



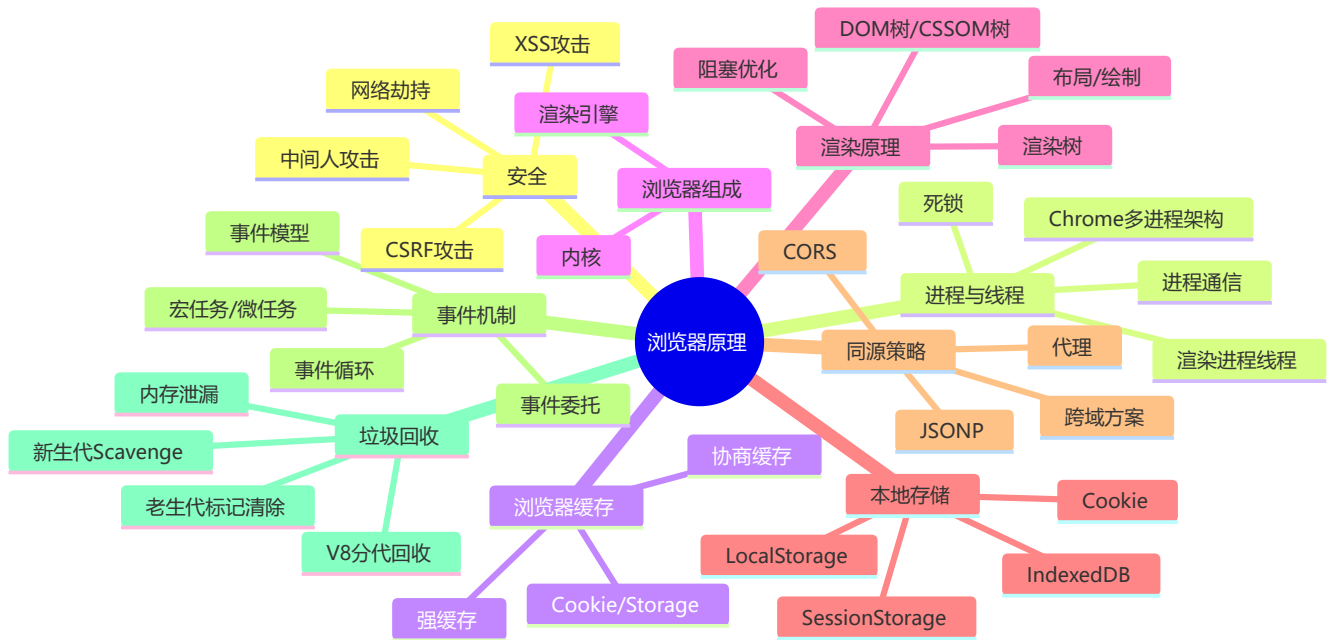
浏览器原理知识详解版（含 Mermaid 图解）

🎯 面试星级：★★★★★ | 建议用时：2 天

浏览器原理核心知识全面梳理，涵盖安全、多进程架构、缓存、渲染流水线、事件机制、V8 GC、bfcache 等核心考点



知识脑图



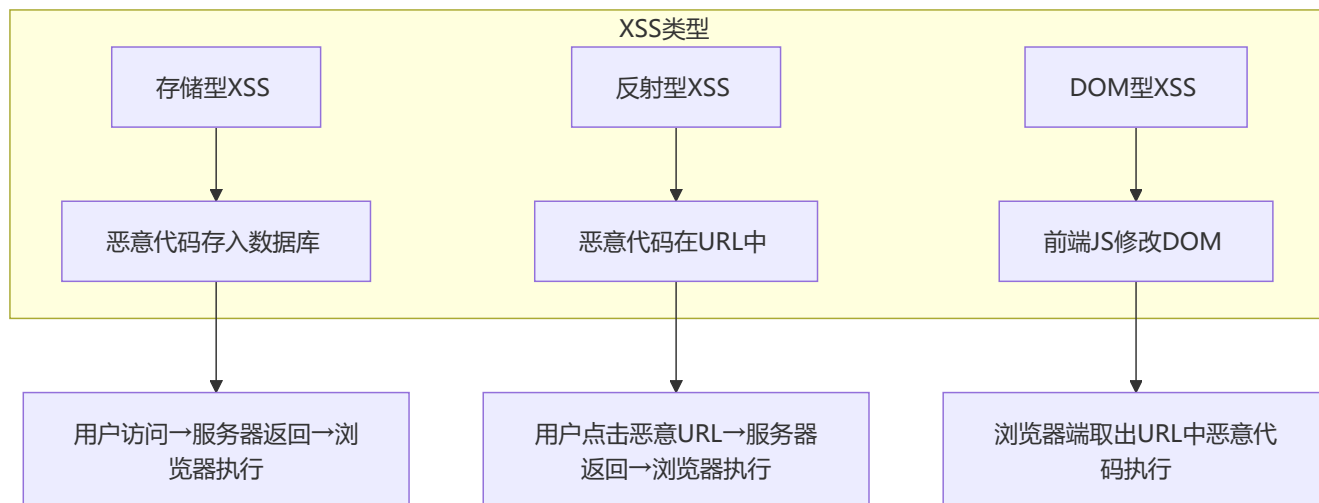
目录

- 🔒 一、浏览器安全
- ⚙️ 二、进程与线程
- 💾 三、浏览器缓存
- 🏠 四、浏览器组成
- 🎨 五、浏览器渲染原理
- 📦 六、浏览器本地存储
- 🌐 七、浏览器同源策略
- 📡 八、浏览器事件机制
- 🗑️ 九、浏览器垃圾回收机制
- 🔍 十、现代浏览器安全新特性
- 🚀 十一、现代浏览器特性
- 📊 十二、Web性能API
- ⚡ 十三、浏览器渲染进阶
- 👤 十四、Web存储进阶
- 🏠 十五、浏览器架构演进

🔒 一、浏览器安全

1 什么是 XSS 攻击？

💡 **要点：** 跨站脚本攻击，攻击者通过注入恶意脚本到网页中执行，从而盗取用户 Cookie 等信息。分存储型、反射型和 DOM 型三种。



XSS 攻击指的是跨站脚本攻击，是一种代码注入攻击。攻击者通过在网站注入恶意脚本，使之在用户的浏览器上运行，从而盗取用户的信息如 cookie 等。

XSS 可以分为存储型、反射型和 DOM 型：

- **存储型：** 恶意脚本会存储在目标服务器上，当浏览器请求数据时，脚本从服务器传回并执行。常见于论坛发帖、商品评论、用户私信等。
- **反射型：** 攻击者诱导用户访问一个带有恶意代码的 URL 后，服务器端接收数据后处理，然后把带有恶意代码的数据发送到浏览器端执行。常见于网站搜索、跳转等。
- **DOM 型：** 通过修改页面的 DOM 节点形成的 XSS。取出和执行恶意代码由浏览器端完成，属于前端 JavaScript 自身的安全漏洞。

2 如何防御 XSS 攻击？

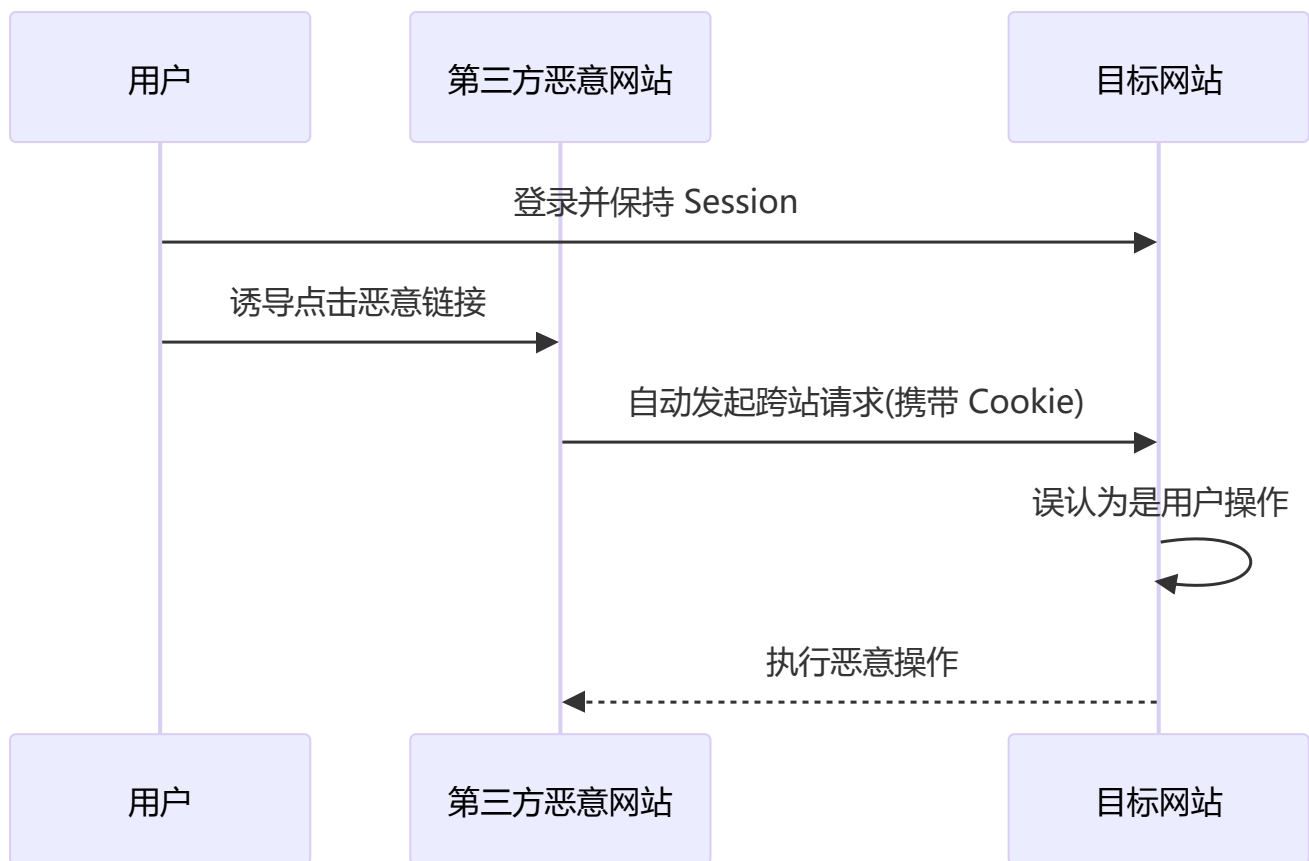
⚠️ **Warning：** XSS 防御是前端安全的第一道防线。永远不要信任用户输入，对所有插入 HTML 的内容进行转义，并通过 CSP 建立资源白名单，对 Cookie 设置 HttpOnly 防止脚本读取。

💡 **要点：** 防御核心三要素：输入输出转义 + CSP 内容安全策略 + HttpOnly Cookie。

- 对需要插入到 HTML 中的代码做好充分的转义
- 使用 CSP（内容安全策略），建立白名单，告诉浏览器哪些外部资源可以加载和执行
- 对敏感信息进行保护，比如 cookie 使用 HttpOnly

3 什么是 CSRF 攻击?

💡 **要点:** 跨站请求伪造, 利用 Cookie 在同源请求中自动携带的特性冒充用户发起恶意请求。



CSRF 攻击指的是跨站请求伪造攻击, 攻击者诱导用户进入一个第三方网站, 然后该网站向被攻击网站发送跨站请求。CSRF 攻击的本质是利用 Cookie 会在同源请求中携带发送给服务器的特点, 以此来实现用户的冒充。

常见的 CSRF 攻击有三种:

- GET 类型的 CSRF 攻击
- POST 类型的 CSRF 攻击
- 链接类型的 CSRF 攻击

4 如何防御 CSRF 攻击?

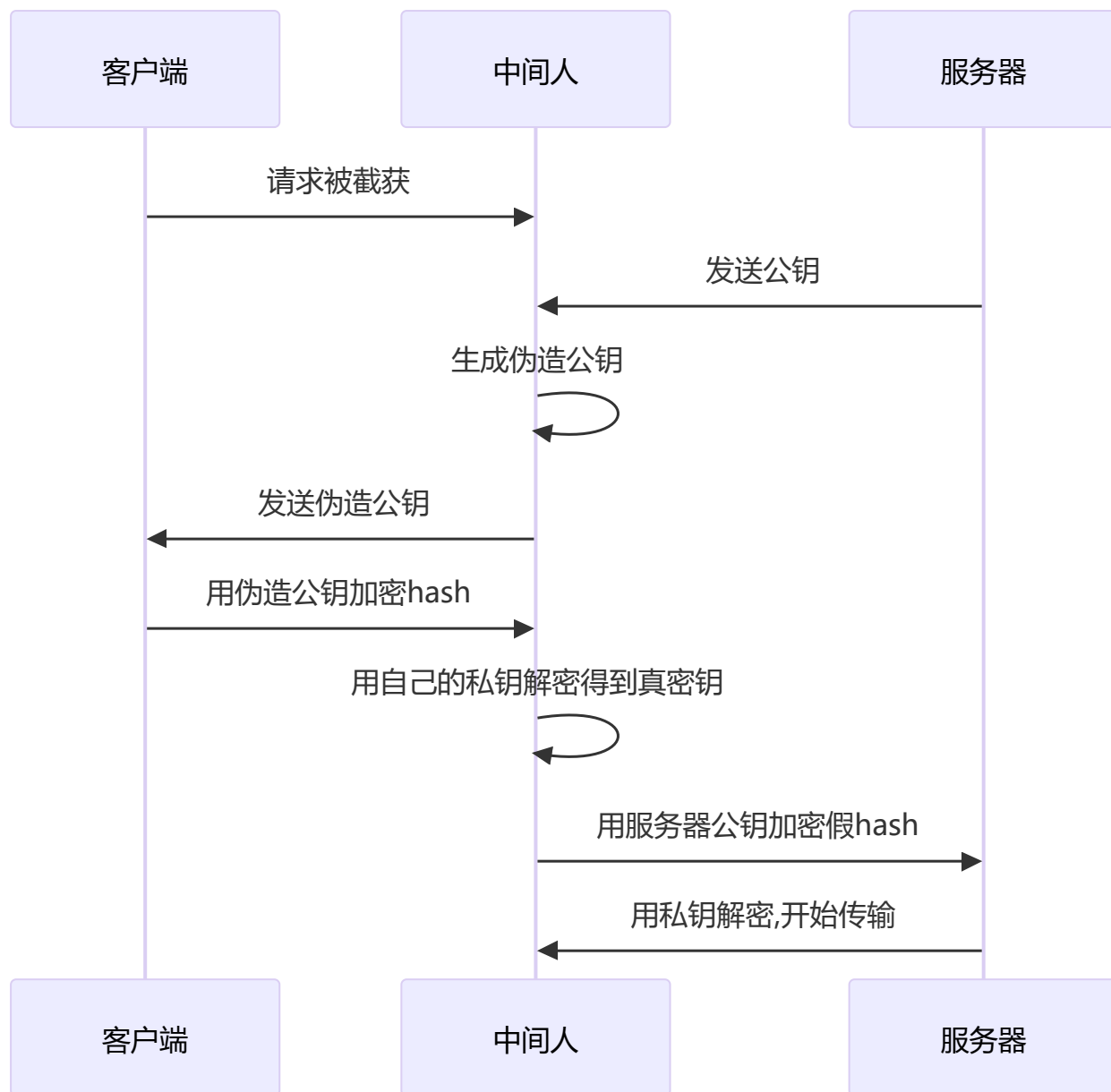
⚠ **Warning:** CSRF 攻击利用 Cookie 自动携带的特性冒充用户, 防御关键在于验证请求来源 (Origin/Referer) 和添加额外的认证信息 (Token)。设置 SameSite 属性是最简单有效的防御手段。

💡 **要点:** 防御方案: 同源检测 + CSRF Token + 双重 Cookie 验证 + SameSite 属性。

- **进行同源检测:** 根据 origin 或 referer 信息判断
- **使用 CSRF Token 进行验证**
- **对 Cookie 进行双重验证**
- **在设置 cookie 属性的时候设置 SameSite**

5 什么是中间人攻击?

💡 要点: 攻击者在客户端和服务端之间拦截通信, HTTPS 可有效防御中间人攻击。



6 网络劫持有哪几种?

💡 要点: DNS 劫持和 HTTP 劫持是最常见的两种, 全站 HTTPS 加密是最有效的防御手段。

- **DNS劫持:** 修改运营商的本地DNS记录, 引导用户流量到缓存服务器
- **HTTP劫持:** 由于HTTP明文传输, 运营商修改http响应内容 (加广告)

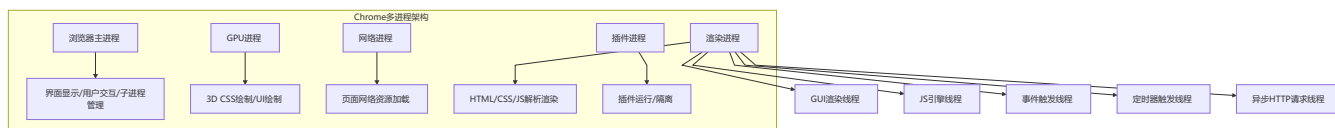
最有效的办法就是全站HTTPS, 将HTTP加密。

🔧 二、进程与线程

1 进程与线程的概念

💡 要点：进程 = 资源分配最小单位，线程 = CPU 调度最小单位。Chrome 采用多进程架构打开一个页面最少需要 4 个进程。

进程是资源分配的最小单位，线程是CPU调度的最小单位。



打开一个网页，最少需要四个进程：1 个网络进程、1 个浏览器进程、1 个 GPU 进程以及 1 个渲染进程。

2 浏览器渲染进程的线程有哪些

💡 要点：渲染进程包含 5 大线程，GUI 渲染线程和 JS 引擎线程互斥，JS 执行时页面渲染被挂起。

- **GUI渲染线程**：负责渲染页面，解析HTML、CSS，构建DOM树、CSSOM树、渲染树和绘制页面
- **JS引擎线程**：负责处理 JavaScript 脚本程序
- **事件触发线程**：控制事件循环
- **定时器触发线程**：setInterval与setTimeout所在线程
- **异步HTTP请求线程**：XMLHttpRequest连接后通过浏览器新开一个线程请求

注意：GUI渲染线程和JS引擎线程是互斥的，当JS引擎执行时GUI线程会被挂起。

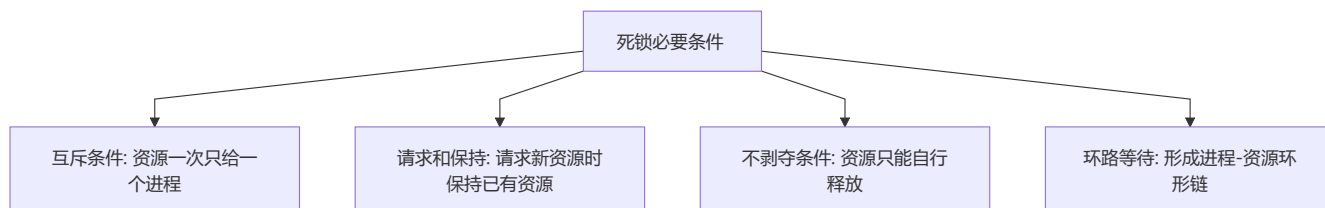
3 进程之间的通信方式

💡 要点：六种 IPC 方式：管道、消息队列、信号量、信号、共享内存、套接字。浏览器进程间常用 IPC 通信。

- **管道通信**：操作系统在内核中开辟的一段缓冲区
- **消息队列通信**：从一个进程向另一个进程发送一个数据块
- **信号量通信**：计数器，实现进程之间的互斥与同步
- **信号通信**：操作系统通过信号来通知进程发生了某种事件
- **共享内存通信**：映射一段能被其他进程所访问的内存
- **套接字通信**：不同主机之间的进程通信

4 死锁产生的原因？

💡 要点：死锁四大必要条件：互斥、请求和保持、不剥夺、环路等待，缺一不可。



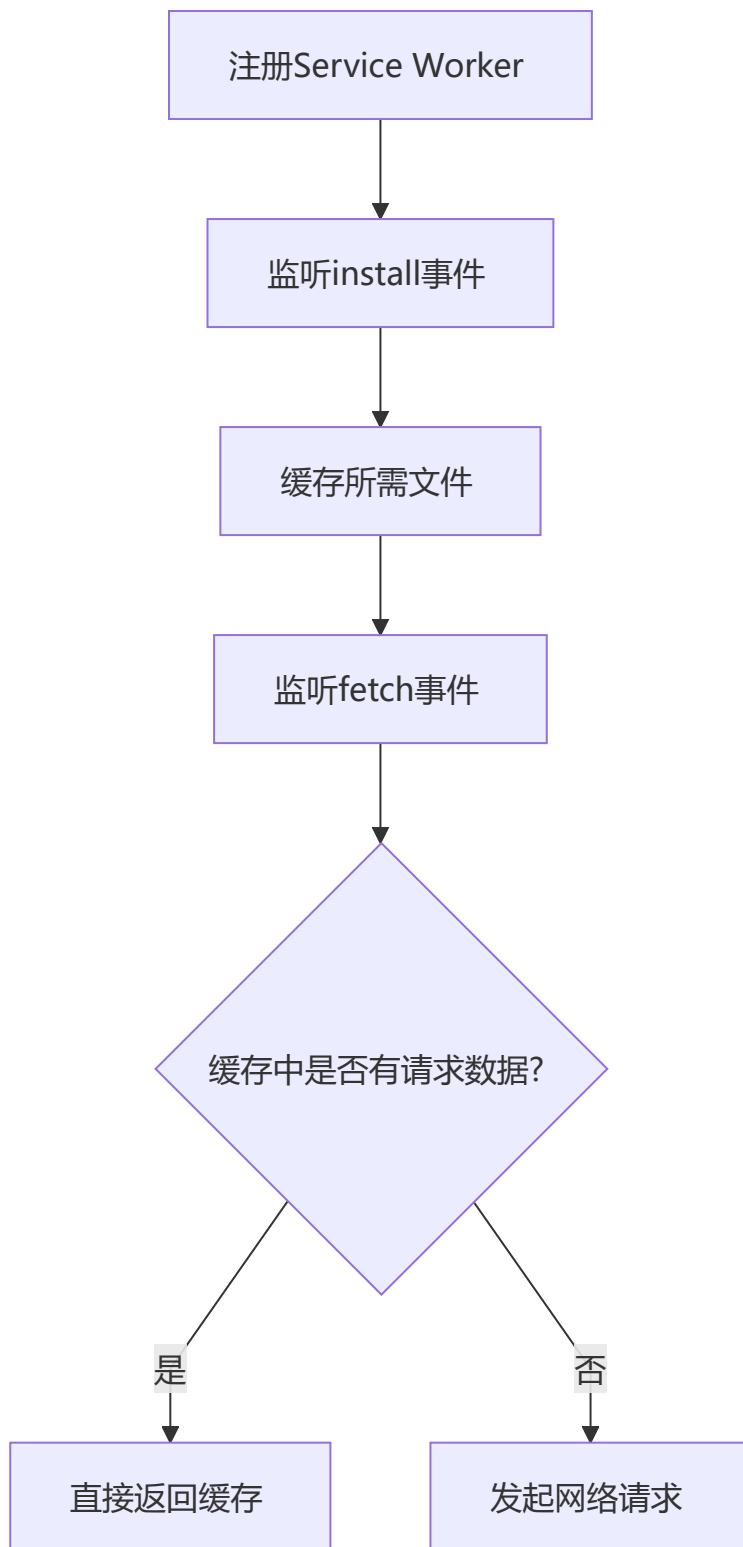
5 如何实现浏览器内多个标签页之间的通信?

💡 要点：四种跨标签页通信方式：WebSocket、SharedWorker、localStorage 事件、postMessage。

- 使用 websocket 协议
- 使用 SharedWorker 的方式
- 使用 localStorage 的方式（监听storage事件）
- 使用 postMessage 方法

6 对Service Worker的理解

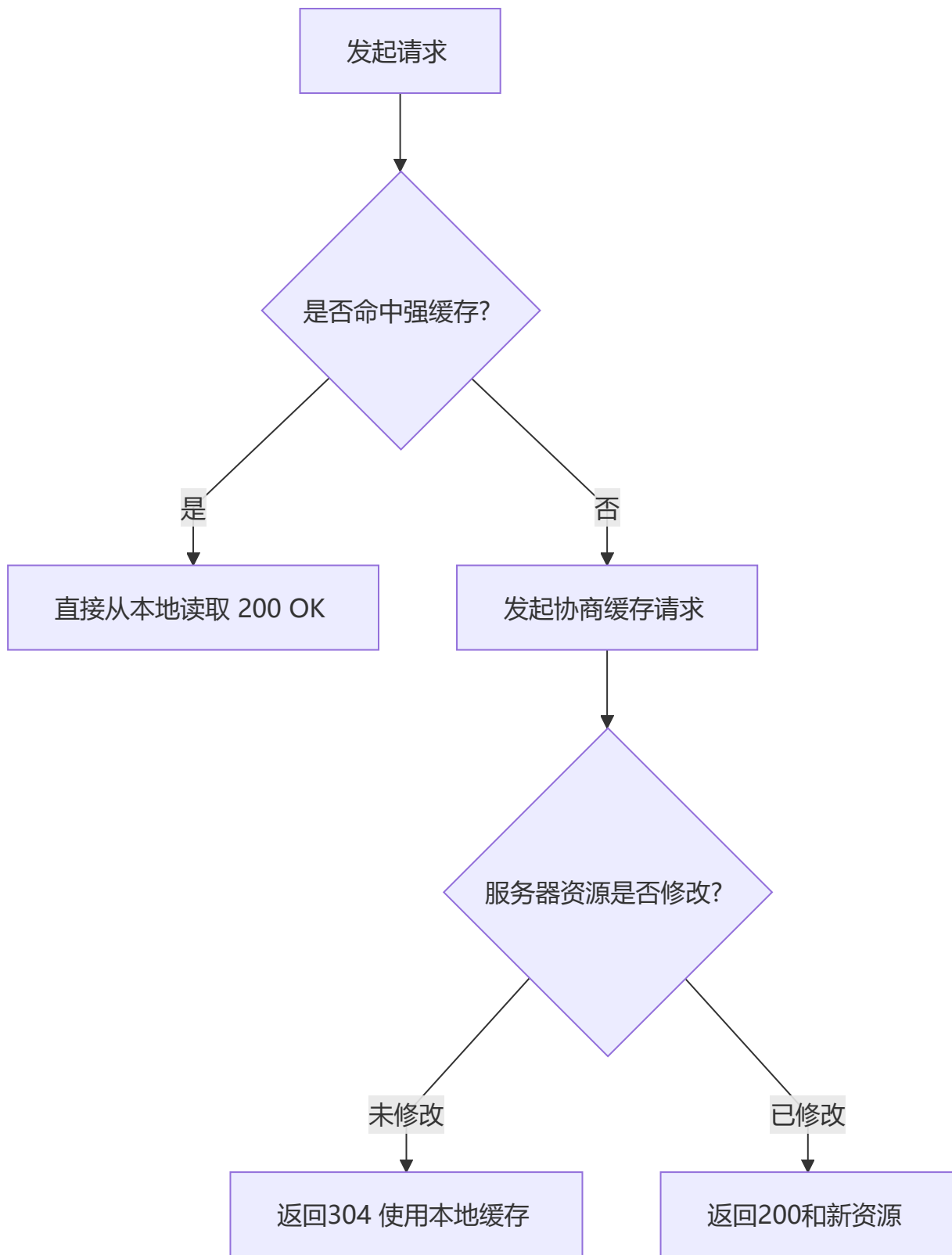
💡 要点：独立于主线程的后台脚本，实现离线缓存、消息推送和后台同步，基于事件驱动。



三、浏览器缓存

1 浏览器缓存机制

💡 **要点：**两级缓存策略：强缓存（Expires/Cache-Control）命中直接使用本地副本，协商缓存（Last-Modified/Etag）需与服务器确认。



 **Note:** 强缓存命中直接返回 200 (from disk/memory cache) , 完全不请求服务器; 协商缓存需要与服务器确认, 资源未修改返回 304。Cache-Control 的 max-age 优先级高于 Expires, Etag 优先级高于 Last-Modified。

强缓存:

- Expires (HTTP1.0) : 绝对时间
- Cache-Control (HTTP1.1) : max-age 相对时间, 优先级高于Expires

协商缓存:

- Last-Modified / If-Modified-Since: 最后修改时间 (精确到秒)
- Etag / If-None-Match: 唯一标识符 (优先级更高)

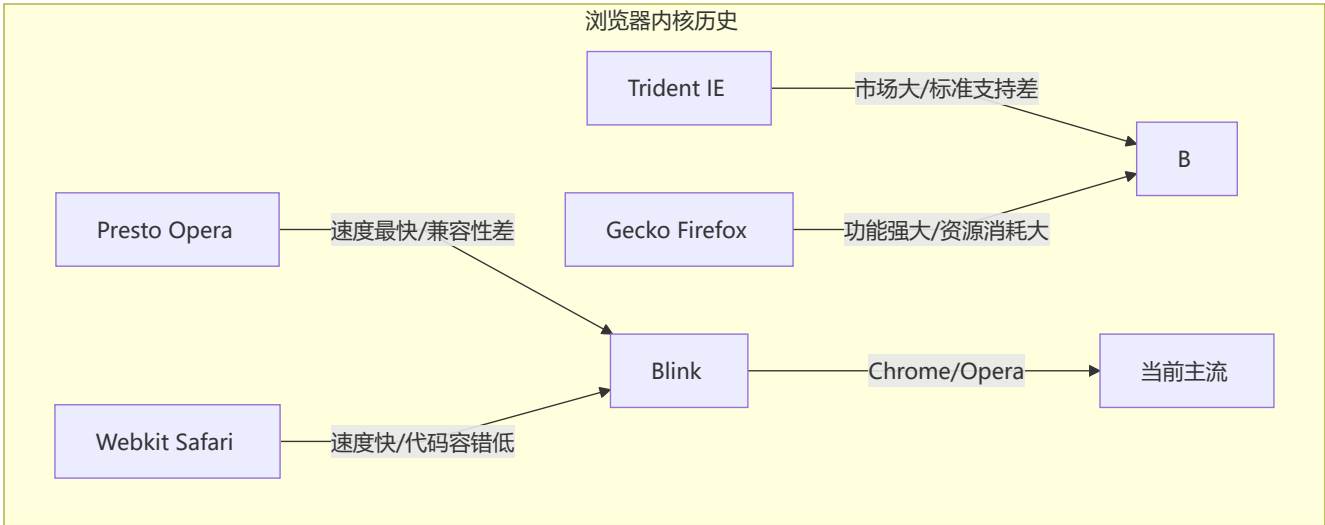
2 点击刷新/强制刷新的区别

- 💡 要点: F5 走协商缓存 (带条件请求头), Ctrl+F5 强制刷新不带任何缓存头, 地址栏回车走完整缓存流程。
- **F5刷新**: 浏览器会带上 If-Modified-Since、If-None-Match, 服务器检查新鲜度, 返回304或200
- **Ctrl+F5强制刷新**: 不会带上条件请求头, 相当于从未请求过, 返回200
- **地址栏回车**: 正常流程, 本地检查是否过期, 服务器检查新鲜度

🏠 四、浏览器组成

1 浏览器内核

- 💡 要点: 不同浏览器内核不同: IE(Trident)、Firefox(Gecko)、Chrome/Opera(Blink)、Safari(Webkit)。



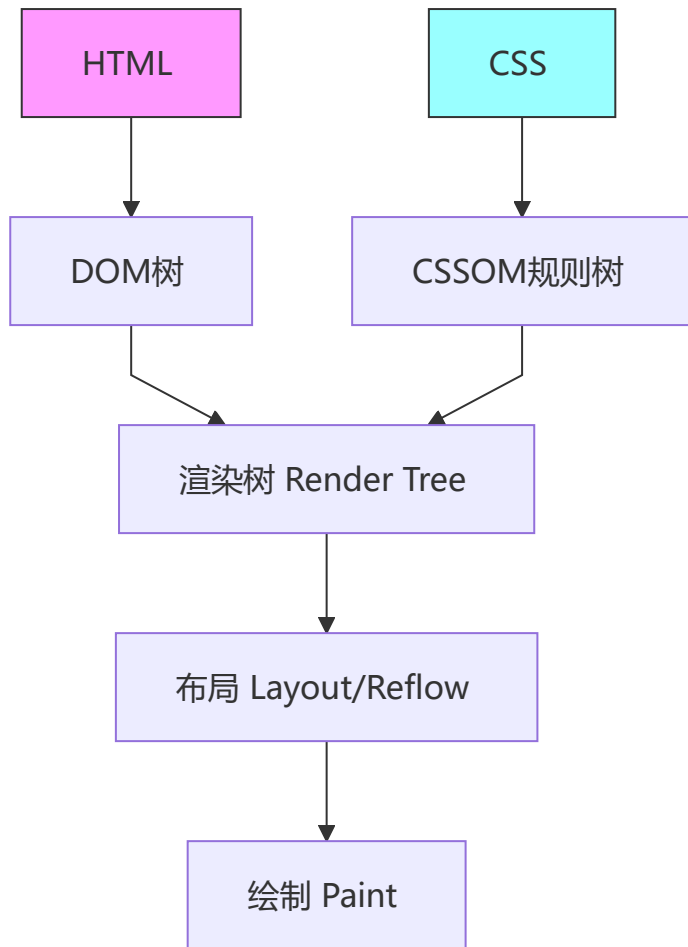
2 浏览器的主要组成部分

- 💡 要点: 浏览器由 7 大组件构成: 用户界面、浏览器引擎、呈现引擎、网络、UI 后端、JS 解释器、数据存储。
- **用户界面**: 地址栏、前进/后退按钮、书签菜单等
- **浏览器引擎**: 在用户界面和呈现引擎之间传送指令
- **呈现引擎**: 负责显示请求的内容
- **网络**: 用于网络调用
- **用户界面后端**: 用于绘制基本的窗口小部件
- **JavaScript 解释器**: 用于解析和执行 JavaScript 代码
- **数据存储**: 持久层, 在硬盘上保存各种数据

🎨 五、浏览器渲染原理

1 浏览器的渲染过程

💡 要点：HTML → DOM 树 + CSS → CSSOM 树 → 渲染树 → 布局 (Layout) → 绘制 (Paint)，整个过程逐步完成，边解析边渲染。



注意：这个过程是逐步完成的，渲染引擎将会尽可能早地将内容呈现到屏幕上，并不会等到所有的html都解析完成之后再去做构建和布局 render 树。

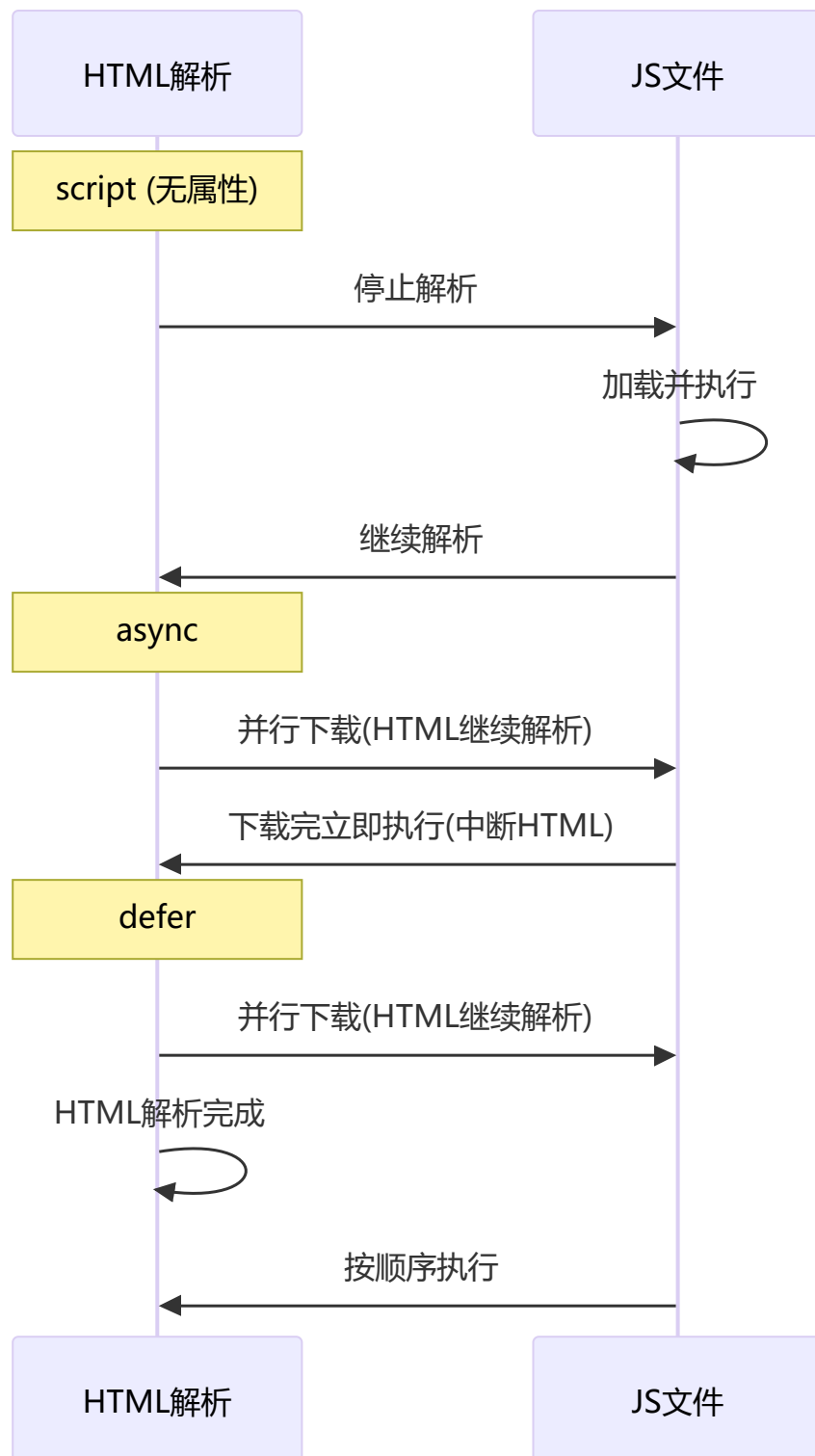
2 浏览器渲染优化

💡 要点：核心优化方向：JS 放 body 底部、CSS 放 head 顶部、减少回流重绘、使用 async/defer 异步加载脚本。

(1) 针对JavaScript:

- 将JavaScript文件放在body的最后
- body中间尽量不要写 `<script>` 标签
- 使用 `async` 或 `defer` 属性异步加载

script、async、defer的区别：



(2) 针对CSS:

- 使用link而不用@import
- CSS一般写在header中

(3) 减少回流与重绘:

- 使用absolute/fixed脱离文档流
- 使用documentFragment
- display: none 后操作

- 读写操作分离

3 渲染过程中遇到 JS 文件如何处理？

💡 **要点：**JavaScript 会阻塞 HTML 解析，遇到 script 标签时暂停 DOM 构建，JS 执行完毕后再恢复解析。

JavaScript 的加载、解析与执行会阻塞文档的解析。如果遇到了 JavaScript，浏览器会暂停文档的解析，将控制权移交给 JavaScript 引擎，等 JavaScript 引擎运行完毕，浏览器再从中断的地方恢复继续解析文档。

4 CSS 如何阻塞文档解析？

💡 **要点：**CSSOM 未完成时阻塞 JS 脚本执行（因为 JS 可能查询样式），间接阻塞 HTML 解析，但不直接阻塞 DOM 构建。

如果浏览器尚未完成 CSSOM 的下载和构建，而我们却想在此时运行脚本，那么浏览器将延迟 JavaScript 脚本执行和文档的解析，直至其完成 CSSOM 的下载和构建。

六、浏览器本地存储

1 存储方式对比

💡 **要点：**Cookie (4KB, 请求自动携带)、LocalStorage (5MB, 持久)、SessionStorage (5MB, 会话级)、IndexedDB (250MB+, NoSQL)。

浏览器存储

Cookie 4KB

请求自动携带

可设置过期时间

同源限制

LocalStorage 5MB

持久存储

不会自动携带

同源限制

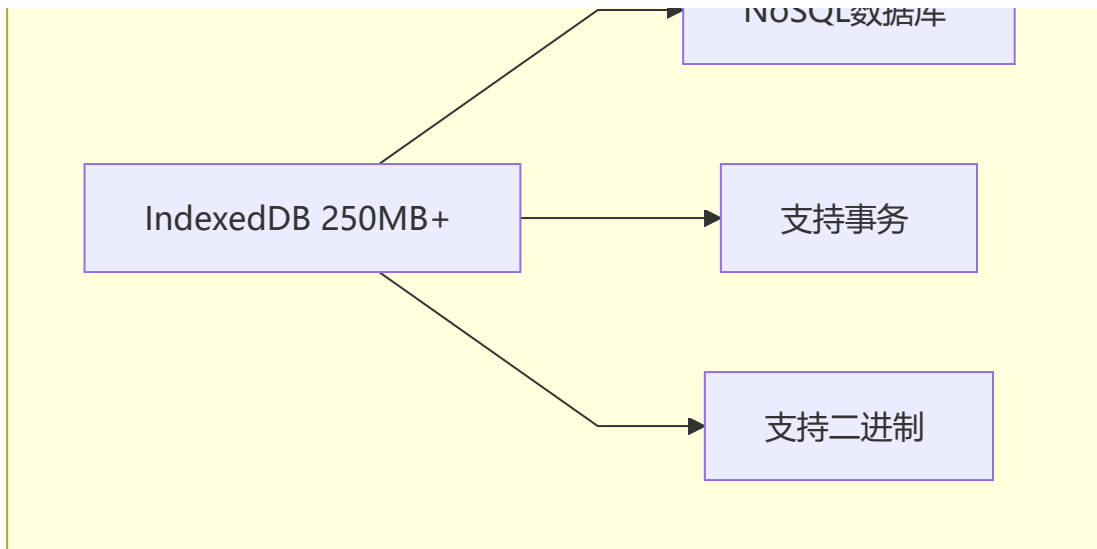
SessionStorage 5MB

会话级存储

同一窗口共享

关闭窗口即清除

IndexedDB 数据库



2 IndexedDB特点

💡 **要点：** NoSQL 数据库，异步操作不锁死浏览器，支持事务和二进制存储，空间大（250MB+）。

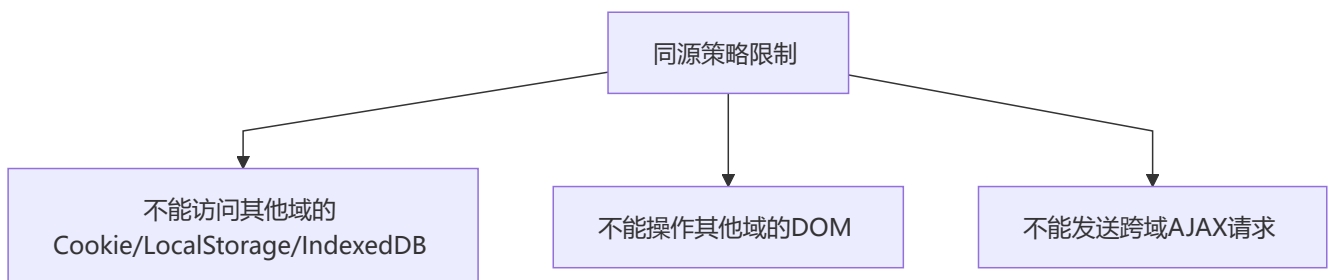
- 键值对存储
- 异步操作，不会锁死浏览器
- 支持事务
- 同源限制
- 存储空间大（不少于250MB）
- 支持二进制存储

🌐 七、浏览器同源策略

1 什么是同源策略

💡 **要点：** 同源 = 协议 + 端口 + 域名三者完全一致。限制跨域 Cookie/DOM/AJAX 访问，是浏览器最核心的安全机制。

同源策略：**协议、端口号、域名**必须一致。



2 如何解决跨域问题

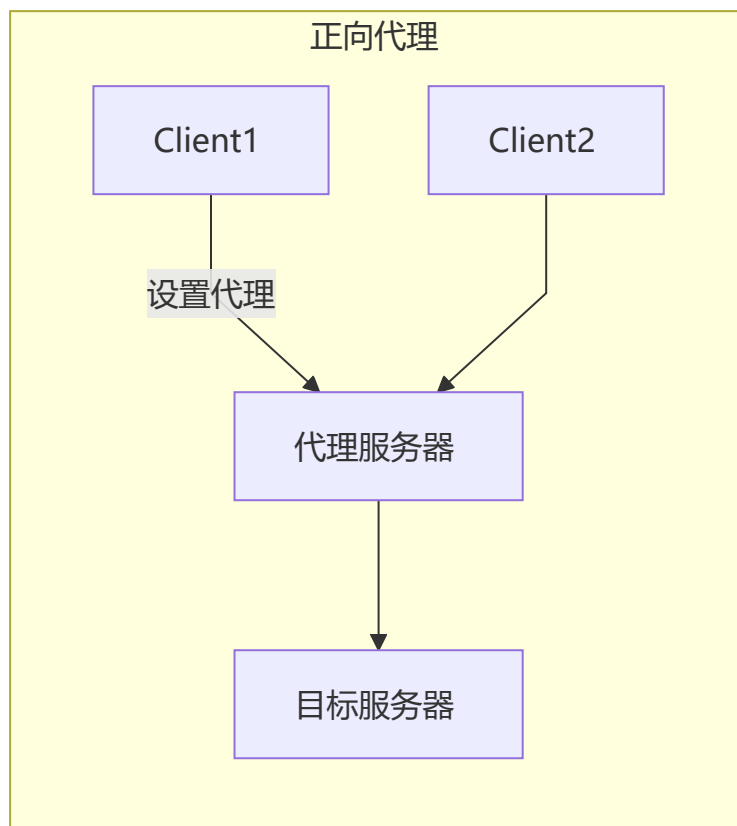
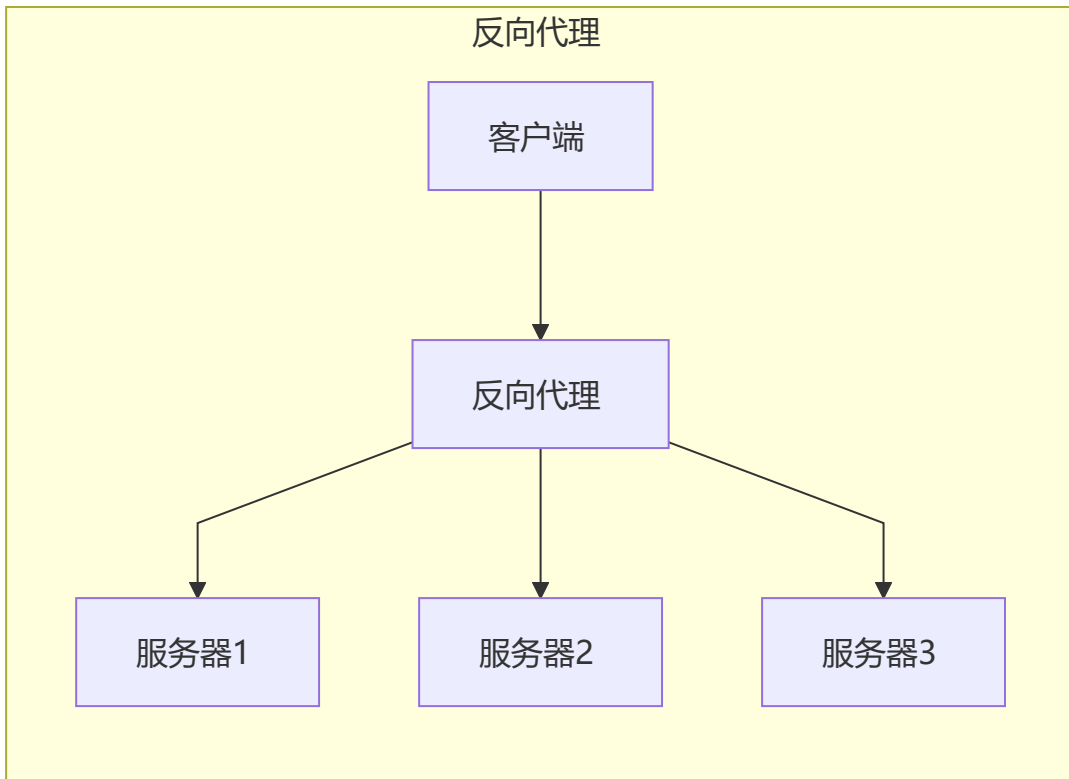
⚠ **Warning:** 跨域是浏览器同源策略的正常安全限制，不要尝试禁用同源策略。CORS 是官方推荐的跨域方案，需要服务端配合设置响应头；开发环境可用 proxy 代理解决。

💡 **要点:** 主要方案：CORS（推荐）、JSONP（仅 GET）、代理转发、postMessage、WebSocket 等。



3 正向代理和反向代理的区别

💡 **要点:** 正向代理隐藏客户端（如 VPN/V2Ray），反向代理隐藏服务器（如 Nginx 负载均衡）。

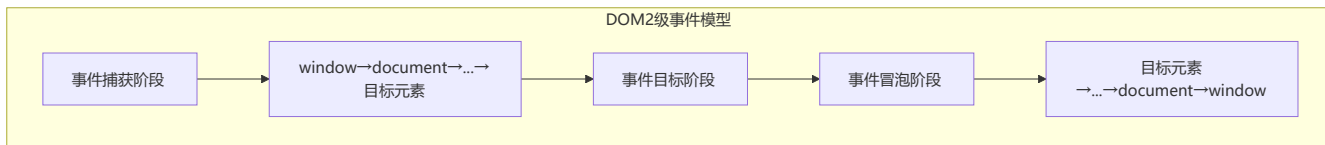


- **正向代理**：客户端设置代理服务器，隐藏客户端（如VPN）
- **反向代理**：服务器设置代理，隐藏真实服务器（如负载均衡）

八、浏览器事件机制

1 事件模型

💡 **要点：** 三种事件模型：DOM0（无传播）、IE（冒泡）、DOM2（捕获→目标→冒泡）。推荐使用 DOM2 级事件模型。



三种事件模型：

- **DOM0级事件模型：** 不会传播，没有事件流概念
- **IE事件模型：** 事件处理阶段 + 事件冒泡阶段
- **DOM2级事件模型：** 事件捕获阶段 + 事件目标阶段 + 事件冒泡阶段

2 事件委托

💡 **要点：** 利用事件冒泡将子元素事件委托给父元素统一处理，减少内存消耗并支持动态绑定。

事件委托本质上是利用了**浏览器事件冒泡**的机制。把子节点的监听函数定义在父节点上，由父节点的监听函数统一处理多个子元素的事件。

优点：

- 减少内存消耗
- 动态绑定事件

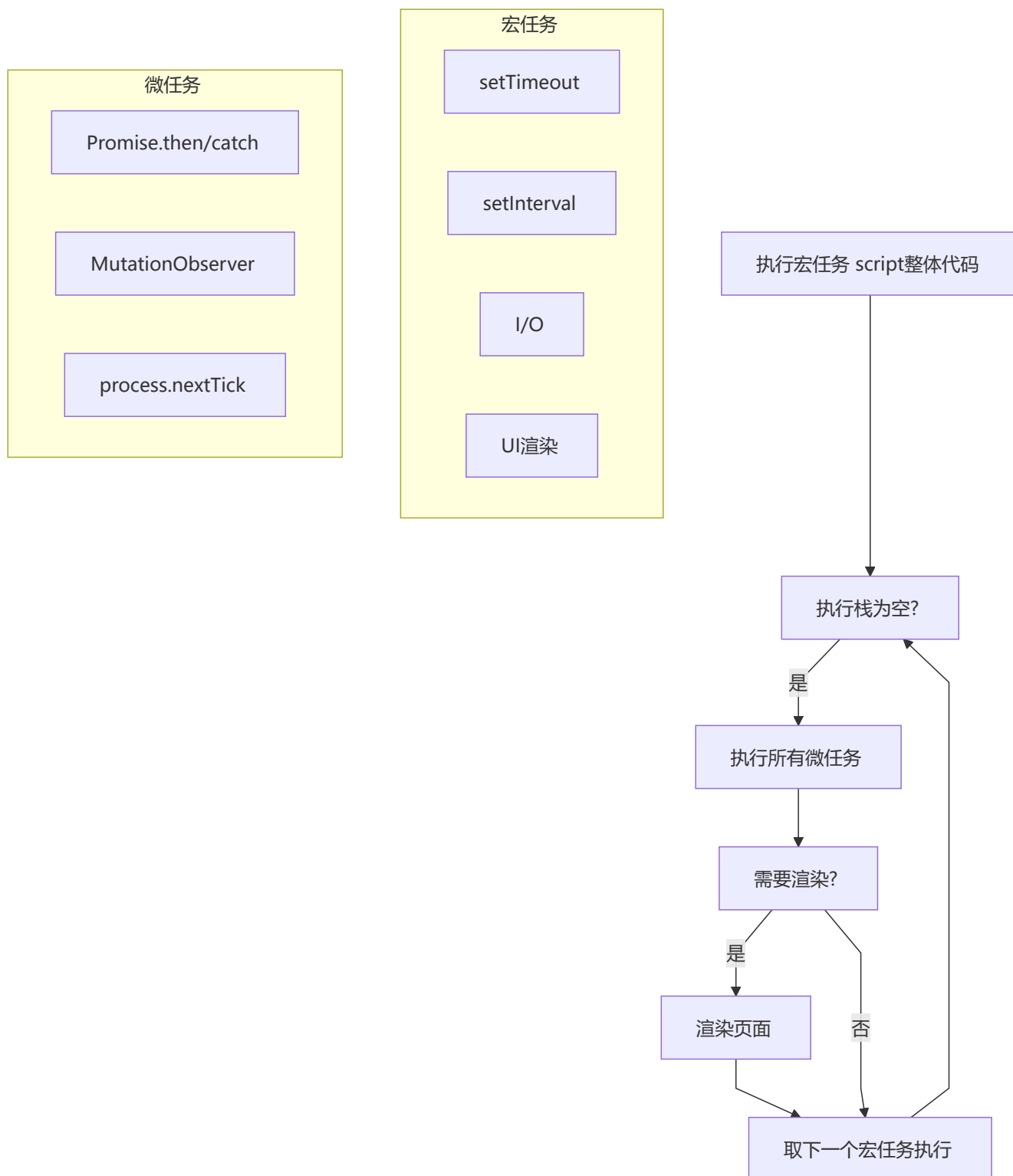
局限性：

- focus、blur 没有事件冒泡机制
- mousemove、mouseout 对性能消耗高

3 事件循环

📄 **Note：** 经典面试题：setTimeout(fn, 0) 的回调需等待当前宏任务和所有微任务完成后才执行。

Promise.then 是微任务，执行时机早于 setTimeout。async/await 的 await 后面的代码相当于 Promise.then 微任务。



4 Node.js 中的 Event Loop

💡 **要点:** Node 的 6 个阶段: Timers → Pending → Idle → Poll → Check → Close。process.nextTick 独立于 Event Loop 且优先级最高。

Node 的 Event Loop 分为 6 个阶段:

1. **Timers (计时器阶段)**: 执行过期的 setTimeout/setInterval 回调
2. **Pending callbacks**: 执行推迟到下一个循环的 I/O 回调
3. **Idle/Prepare**: 仅供内部使用

4. **Poll (轮询阶段)**：获取新的 I/O 事件
5. **Check (查询阶段)**：执行 setImmediate 回调
6. **Close callbacks**：执行关闭回调

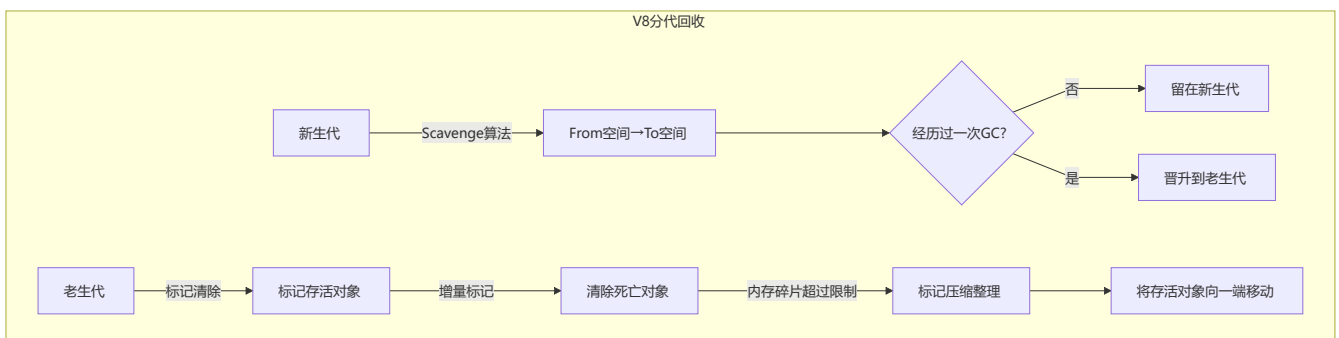
💡 **Tip**：Node.js 中 `process.nextTick` 优先级高于所有微任务，在 Event Loop 每个阶段完成后立即执行，使用不当可能导致 I/O 饥饿。

`process.nextTick` 独立于 Event Loop 之外，每个阶段完成后清空 nextTick 队列，优先于其他微任务执行。

🗑️ 九、浏览器垃圾回收机制

1 V8的垃圾回收机制

💡 **要点**：V8 分代回收：新生代用 Scavenge 算法（From→To 复制），老生代用标记清除/压缩算法。从 stop-the-world 到并发标记逐步优化。



新生代算法 (Scavenge)：内存分为 From 和 To 两个空间，From 满时，将存活对象复制到 To 空间，然后互换 From 和 To。

老生代算法：

- 标记清除算法 (Mark-Sweep)：标记存活对象，销毁未标记对象
- 标记压缩算法 (Mark-Compact)：将存活对象向一端移动，清理碎片

V8 从 stop-the-world → 增量标记 → 并发标记，逐步优化GC性能。

2 哪些操作会造成内存泄漏？

💡 **要点**：四种常见泄漏：意外全局变量、未清除的定时器、DOM 引用未释放、不合理的闭包使用。

- 未声明的变量意外创建全局变量
- 设置了 setInterval 定时器忘记取消
- 获取 DOM 元素的引用后被删除，但引用未释放
- 不合理的使用闭包

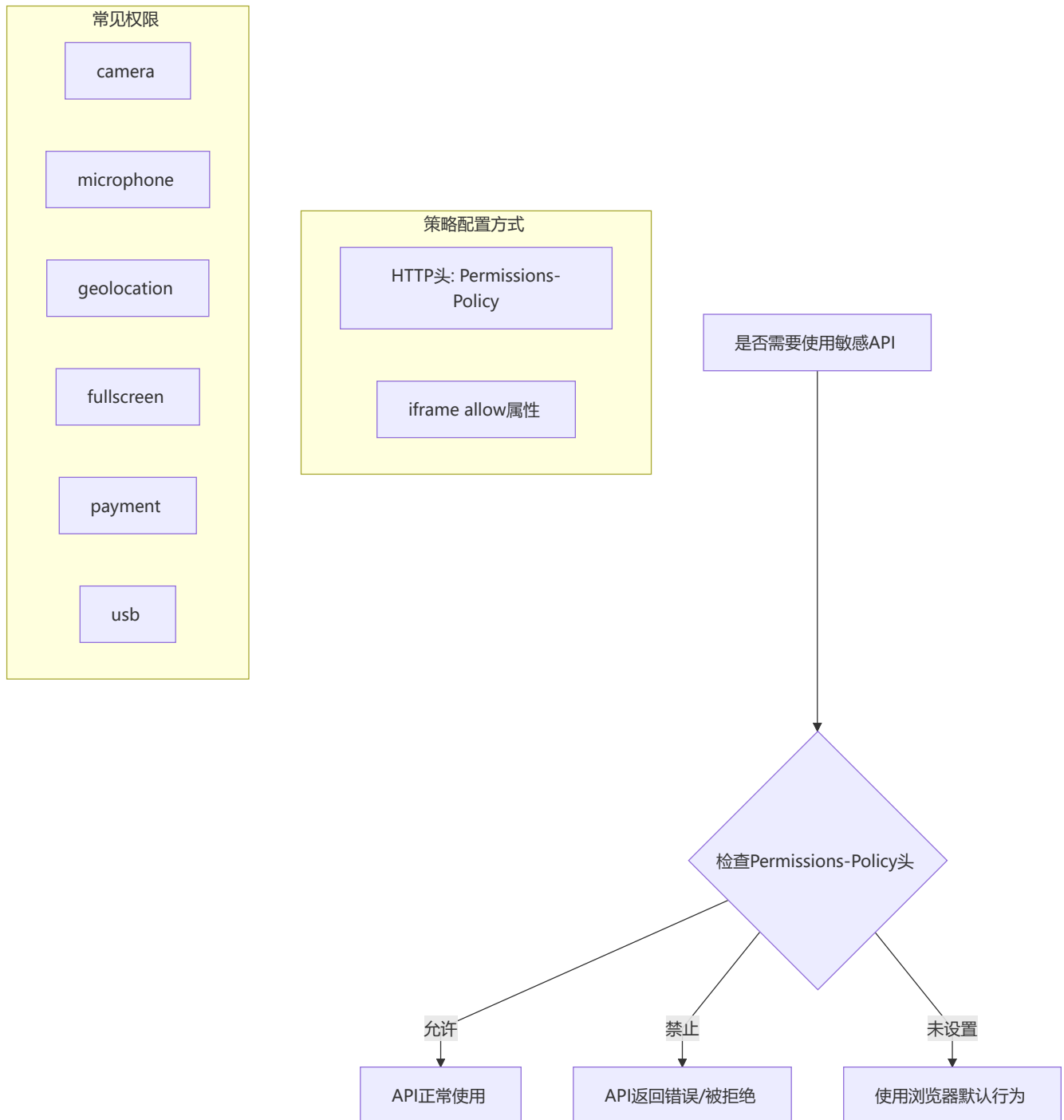
⚠️ **Warning**：内存泄漏会导致页面越来越卡顿甚至崩溃。使用 Chrome Memory 面板的 Heap Snapshot 定期检查 Detached DOM 节点，使用 `performance.measureUserAgentSpecificMemory()` 监控内存变化。

十、现代浏览器安全新特性

1 Permissions Policy (权限策略)

💡 **要点：**通过 HTTP 头或 iframe 属性细粒度控制 API 权限（camera/mic/geolocation 等），比 Feature Policy 更简洁安全。

Permissions Policy (原 Feature Policy) 允许站点控制哪些浏览器特性可以被使用，提供一套细粒度的权限控制机制。



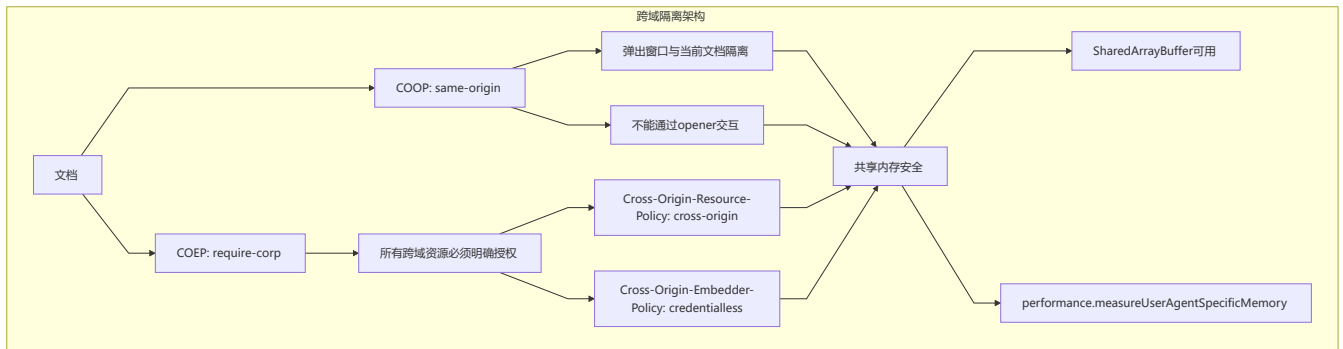
- **HTTP头方式：** `Permissions-Policy: camera=(), microphone=(self), geolocation=(self) "https://trusted-site.com"`
- **iframe方式：** `<iframe allow="camera 'none'; microphone 'self'" src="...">`
- **默认值：**大多数权限默认为 `*` (允许所有) 或 `self` (仅允许同源)

- **与Feature Policy的区别**：语法更简洁，默认行为更安全（默认关闭）

2 Cross-Origin Isolation (跨域隔离)

💡 **要点**：COOP(same-origin) + COEP(require-corp) 实现跨域隔离，防御 Spectre 侧信道攻击，启用 SharedArrayBuffer 的前提。

跨域隔离通过 COOP + COEP 两个 HTTP 头实现，是启用 SharedArrayBuffer 等高性能API的前提。



- **COOP (Cross-Origin Opener Policy)**:
 - `same-origin`：将文档从浏览器的跨域共享上下文中隔离
 - `same-origin-allow-popups`：允许弹出窗口保留引用
 - `unsafe-none`：默认值，不隔离
- **COEP (Cross-Origin Embedder Policy)**:
 - `require-corp`：要求所有跨域资源设置 CORP 头或 CORS
 - `credentialless`：跨域请求不携带凭据(新方案)
- **为何需要**：Spectre漏洞利用共享内存进行侧信道攻击，跨域隔离限制了共享内存的可用性
- **状态检测**：`crossOriginIsolated` (布尔值) 在 window 和 WorkerGlobalScope 上可用

3 Trusted Types (可信类型)

💡 **要点**：禁止 innerHTML/document.write/eval 接收字符串，强制使用安全类型化对象，从根本上防御 DOM XSS，需配合 CSP 使用。

Trusted Types 通过禁止危险的 DOM 操作函数接收字符串，强制开发者使用安全的类型化对象，从根本上防御 DOM XSS。

```
// 配置 CSP 开启 Trusted Types
// Content-Security-Policy: require-trusted-types-for 'script'

// 创建 Trusted Type 策略
const sanitizer = trustedTypes.createPolicy('my-policy', {
  createHTML: (input) => {
    return input.replace(/<script[^\>]*>[\s\S]*?</script>/gi, '')
      .replace(/on\w+="["']*"/gi, '');
  },
  createScriptURL: (url) => {
    const allowed = ['https://cdn.example.com/'];
    if (allowed.some(prefix => url.startsWith(prefix))) {
```

```

    return url;
  }
  throw new TypeError('URL not allowed');
}
});

// 使用 Trusted Type
el.innerHTML = sanitizer.createHTML(userInput); // ✓ 安全
// el.innerHTML = userInput; // ✗ CSP 阻止, 报错

// 默认策略 (兜底)
document.body.innerHTML = trustedTypes.defaultPolicy.createHTML('<h1>Safe</h1>');

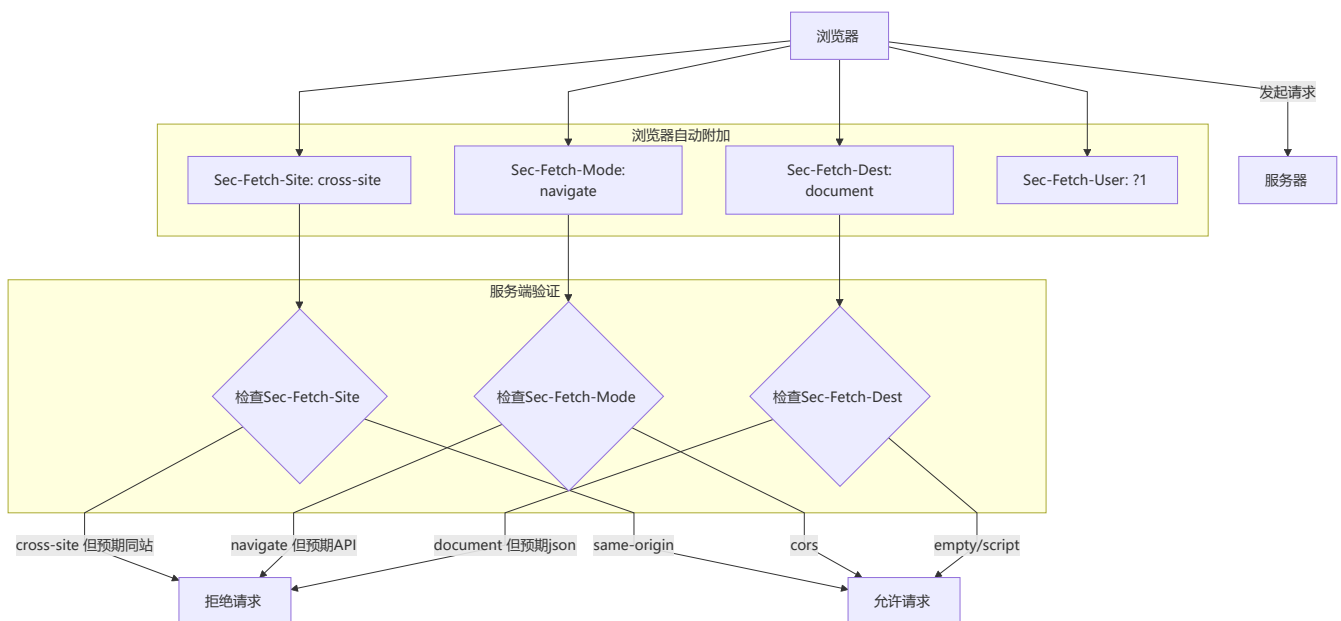
```

- **防御原理:** 将 `innerHTML`、`document.write`、`eval` 等 XSS 高危操作拦截
- **与 CSP 结合:** `require-trusted-types-for 'script'` 开启强制模式
- **报告模式:** `require-trusted-types-for 'script'; report-uri /csp-report`
- **浏览器兼容性:** Chrome 83+, 其他浏览器逐步支持

4 Fetch Metadata (请求元数据)

💡 **要点:** 浏览器自动附加 `Sec-Fetch-*` 请求头, 服务器据此判断请求是否合法, 有效防御 CSRF 和跨域攻击。

Fetch Metadata 通过 `Sec-Fetch-*` 请求头, 让服务器了解请求的来源和上下文, 有效防御跨域攻击。



Sec-Fetch-* 请求头说明:

请求头	含义	示例值
<code>Sec-Fetch-Site</code>	请求发起者与目标的关系	<code>cross-site</code> , <code>same-site</code> , <code>same-origin</code> , <code>none</code>
<code>Sec-Fetch-Mode</code>	请求模式	<code>navigate</code> , <code>cors</code> , <code>no-cors</code> , <code>websocket</code>
<code>Sec-Fetch-Dest</code>	请求目标资源类型	<code>document</code> , <code>script</code> , <code>image</code> , <code>empty</code> , <code>audioworklet</code>

请求头	含义	示例值
<code>Sec-Fetch-User</code>	是否由用户操作触发	?1 (是), ?0 (否)

服务端验证策略示例：

```
// Node.js 中间件示例
app.use((req, res, next) => {
  const site = req.headers['sec-fetch-site'];
  const mode = req.headers['sec-fetch-mode'];

  // 非导航请求禁止跨站
  if (site === 'cross-site' && mode !== 'navigate') {
    return res.status(403).send('Forbidden');
  }

  // API端点只允许同源请求
  if (req.path.startsWith('/api') && site !== 'same-origin') {
    return res.status(403).send('Cross-origin API calls not allowed');
  }

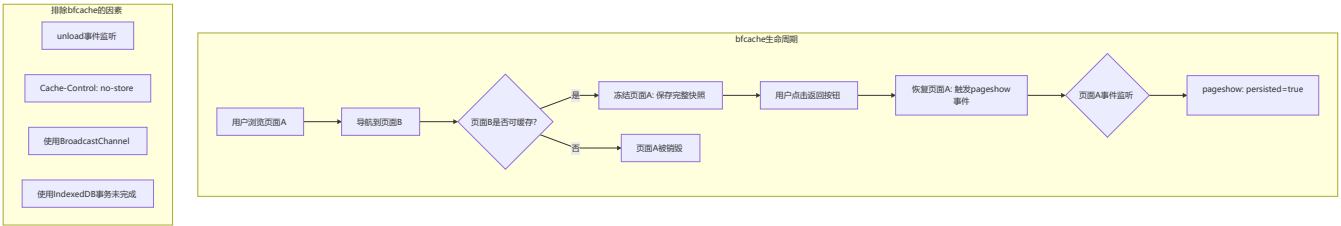
  next();
});
```

十一、现代浏览器特性

1 Back/Forward Cache (bfcache)

要点：前进/后退时保存完整页面快照（含 JS 内存状态），恢复速度极快。避免使用 `unload` 事件和 `Cache-Control: no-store` 以命中 bfcache。

bfcache 是浏览器的一种缓存机制，当用户导航离开页面时，浏览器会保存当前页面的完整快照（包括内存中的 JavaScript 状态），以便用户使用前进/后退按钮时能立即恢复。



与 HTTP Cache 的区别：

特性	bfcache	HTTP Cache
缓存内容	页面完整快照（HTML+JS+CSS+内存状态）	单个资源文件
恢复速度	立即（0ms JS执行）	需要重新解析和执行
触发条件	前进/后退导航	基于缓存策略的任意导航

特性	bfcache	HTTP Cache
生命周期	会话级别	由 Cache-Control 控制

如何测试 bfcache 命中：

```

window.addEventListener('pageshow', (event) => {
  if (event.persisted) {
    console.log('页面从 bfcache 恢复');
    // 重新获取数据或刷新状态
    refreshData();
  }
});

window.addEventListener('pagehide', (event) => {
  if (!event.persisted) {
    console.log('页面被销毁，不会进入 bfcache');
  }
});

```

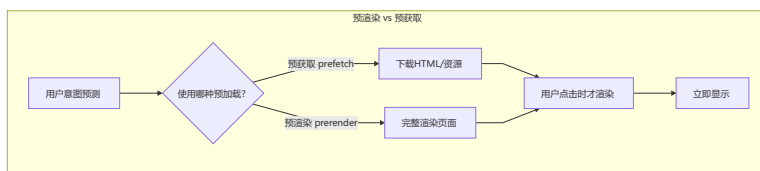
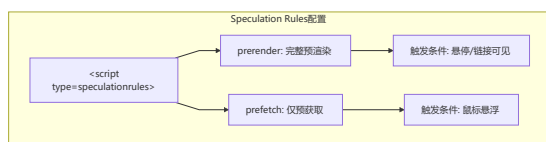
避免被 bfcache 排除的关键点：

- 不监听 `unload` 事件（改为使用 `pagehide`）
- 不使用 `Cache-Control: no-store`
- 在 `pagehide` 中关闭未完成的网络请求和 IndexedDB 事务
- 使用 `navigator.sendBeacon()` 发送分析数据

2 Prerendering (预渲染)

💡 **要点：** 在后台完整渲染页面，用户点击即时显示。Speculation Rules API 提供 `list` 和 `document` 两种预渲染源，`eagerness` 控制触发积极程度。

预渲染允许浏览器在后台完全加载并渲染页面，用户点击时立即显示。



Speculation Rules API (新版预渲染)：

```

<script type="speculationrules">
{
  "prerender": [
    {
      "source": "list",
      "urls": ["/next-page", "/popular-page"],
      "requires": ["anonymous-client-ip-when-cross-origin"]
    }
  ],

```



```
"prefetch": [
  {
    "source": "document",
    "eagerness": "moderate"
  }
]
}
</script>
```

预渲染触发条件：

- `source: "list"` — 指定预渲染的 URL 列表
- `source: "document"` — 根据用户行为（悬停）动态推测
- `eagerness: immediate` (立即) / `eager` (积极) / `moderate` (适度) / `conservative` (保守)

预渲染与预获取的区别：

特性	Prerender	Prefetch
执行程度	完整渲染（HTML解析+CSS+JS执行）	仅下载资源
资源消耗	高（CPU+内存）	低
显示速度	立即（已渲染完成）	需等待解析执行
适用场景	用户大概率访问的页面	可能访问的页面

检测预渲染：

```
if (document.prerendering) {
  console.log('页面正在预渲染');
}

document.addEventListener('prerenderingchange', () => {
  console.log('预渲染完成，页面已激活');
});
```

3 Priority Hints (优先级提示)

💡 **要点：**通过 `fetchpriority="high/low"` 提示浏览器资源优先级，LCP 图片设为 `high` 可显著提升 Largest Contentful Paint 指标。

Priority Hints 通过 `fetchpriority` 属性允许开发者提示浏览器资源的相对优先级，优化关键资源的加载顺序。

默认优先级计算

CSS/字体: 最高

同步脚本: 高

图片(视口内): 中

图片(视口外): 低

异步脚本: 低

资源请求

浏览器计算默认优先级

fetchpriority属性?

high

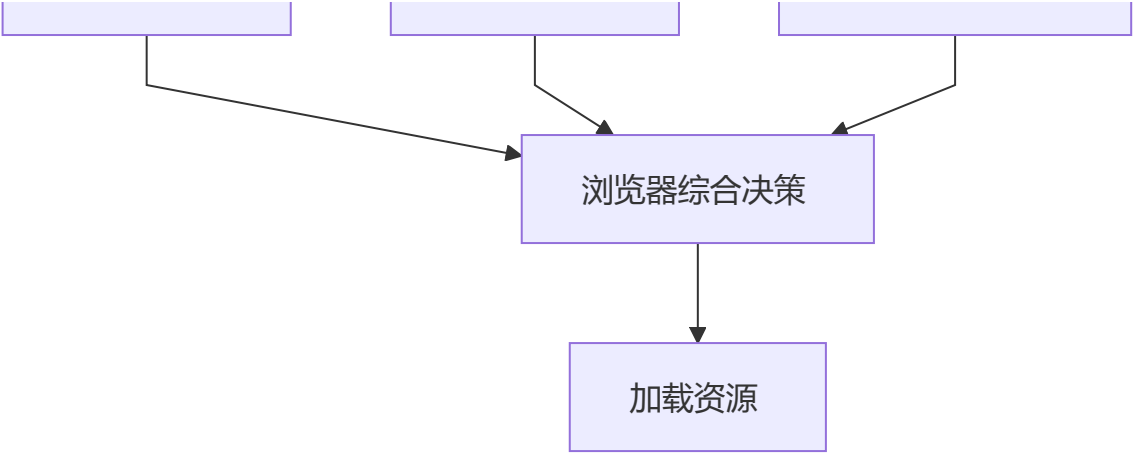
提升优先级

low

降低优先级

auto/无

使用默认优先级



使用示例：

```
<!-- LCP 图片提升优先级 -->


<!-- 非关键图片降低优先级 -->


<!-- 预加载关键字体 -->
<link rel="preload" href="font.woff2" as="font" fetchpriority="high">

<!-- 脚本优先级控制 -->
<script src="critical.js" fetchpriority="high"></script>
<script src="analytics.js" fetchpriority="low" async></script>
```

浏览器优先级策略：

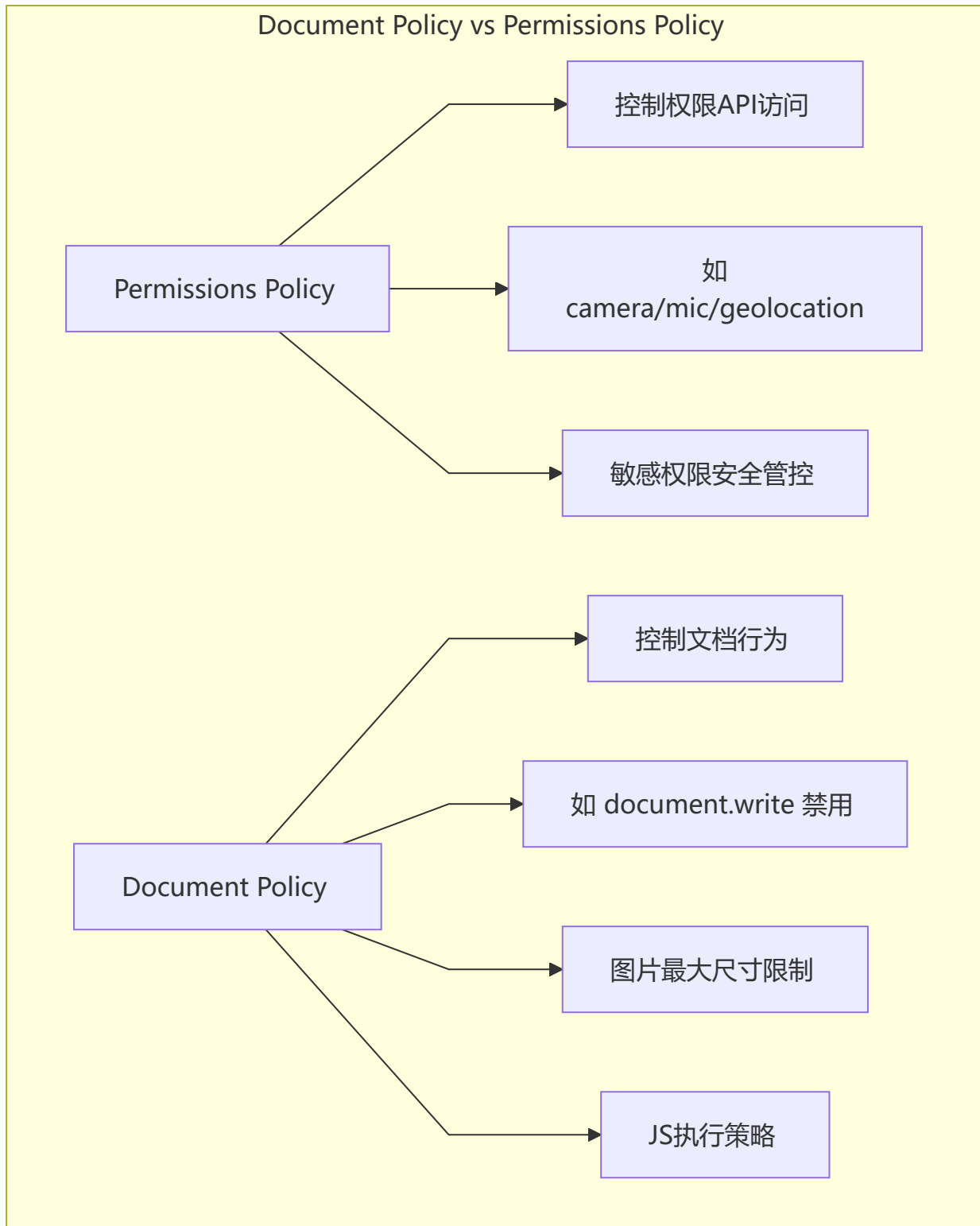
资源类型	默认优先级	可调整
CSS (head中)	Highest	high
字体文件	High	high/low
同步脚本	High	high/low
视口内图片	Medium	high/low
视口外图片	Low	high
异步脚本/import	Low	high/low
预加载(preload)	等同目标类型	high/low

影响: LCP 图片设置 `fetchpriority="high"` 可以显著提升 Largest Contentful Paint 指标。

4 Document Policy

💡 **要点：**控制文档行为而非权限：可禁用 `document.write`、限制图片最大尺寸、禁止 JS 性能分析等，通过 HTTP 头或 `iframe` 属性配置。

Document Policy 控制文档的行为特征（非权限类），如是否允许 `document.write`、图片尺寸限制等。



使用方式：

HTTP 头方式

Document-Policy: document-write=?0, max-image-size=2000

Document-Policy: js-profiling=?1

Document-Policy: force-load-at-top=?0

<!-- iframe 方式 -->

<iframe document-policy="document-write=?0" src="...">

策略配置示例：

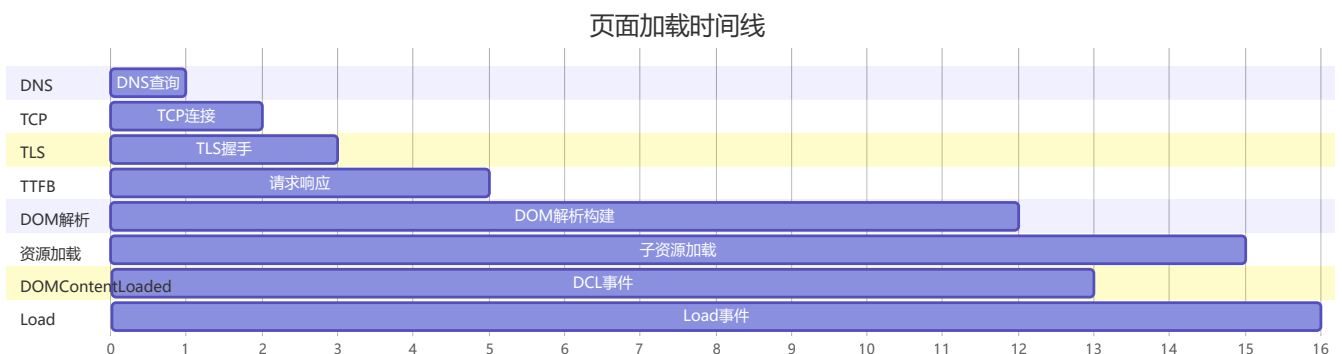
策略名	说明	值
document-write	是否允许 document.write	?0 (禁止) / ?1 (允许)
max-image-size	图片最大尺寸(kb或wpx)	数值 / -1 (不限制)
max-parsed-style-sheet-length	样式表最大长度	数值
js-profiling	是否允许性能分析	?0 / ?1
force-load-at-top	是否强制页面从头加载	?0 / ?1

十二、Web性能API

1 Navigation Timing API

💡 **要点：**精确测量 DNS/TCP/TLS/TTFB/DOM 解析等各阶段耗时，推荐使用 PerformanceNavigationTiming 替代废弃的 performance.timing。

Navigation Timing 提供精确的页面加载各阶段耗时数据，分析性能瓶颈的核心工具。



传统方式 (已废弃)：

```

const timing = performance.timing;
const dns = timing.domainLookupEnd - timing.domainLookupStart;
const tcp = timing.connectEnd - timing.connectStart;
const ttfb = timing.responseStart - timing.requestStart;
const domContentLoaded = timing.domContentLoadedEventEnd - timing.navigationStart;
const pageLoad = timing.loadEventEnd - timing.navigationStart;

```

现代方式 (PerformanceNavigationTiming):

```

const [navEntry] = performance.getEntriesByType('navigation');

// 关键指标
const dns = navEntry.domainLookupEnd - navEntry.domainLookupStart;
const tcp = navEntry.connectEnd - navEntry.connectStart;
const tls = navEntry.secureConnectionStart ?
    navEntry.connectEnd - navEntry.secureConnectionStart : 0;
const ttfb = navEntry.responseStart - navEntry.requestStart;
const domParse = navEntry.domComplete - navEntry.domInteractive;
const dcl = navEntry.domContentLoadedEventEnd - navEntry.domContentLoadedEventStart;
const load = navEntry.loadEventEnd - navEntry.loadEventStart;

const transferSize = navEntry.transferSize; // 总传输字节
const encodedBodySize = navEntry.encodedBodySize; // 编码后大小
const decodedBodySize = navEntry.decodedBodySize; // 解码后大小
const compressed = encodedBodySize < decodedBodySize; // 是否压缩

console.table({
  'DNS解析': `${dns}ms`,
  'TCP连接': `${tcp}ms`,
  'TLS握手': `${tls}ms`,
  'TTFB': `${ttfb}ms`,
  'DOM解析': `${domParse}ms`,
  'DCL': `${dcl}ms`,
  '完全加载': `${load}ms`,
  '压缩比': `${(encodedBodySize / decodedBodySize * 100).toFixed(1)}%`
});

```

2 Resource Timing API

💡 **要点:** 监控所有资源（图片/脚本/样式/XHR）的加载性能，可分析慢资源、统计类型分布、找出最大资源。

Resource Timing API 监控所有页面资源的加载性能，包括图片、脚本、样式表、XHR请求等。

```

// 获取所有资源加载信息
const resources = performance.getEntriesByType('resource');

// 分析慢资源
const slowResources = resources.filter(r => {
  const totalTime = r.responseEnd - r.startTime;
  return totalTime > 1000; // 超过1秒
});

```

```

// 各资源阶段耗时分析
resources.forEach(resource => {
  const { name, initiatorType, duration, transfersSize } = resource;
  const dns = resource.domainLookupEnd - resource.domainLookupStart;
  const tcp = resource.connectEnd - resource.connectStart;
  const ttfb = resource.responseStart - resource.requestStart;

  if (duration > 500) {
    console.warn(`慢资源: ${name}`, {
      类型: initiatorType,
      总耗时: `${duration.toFixed(2)}ms`,
      DNS: `${dns.toFixed(2)}ms`,
      TCP: `${tcp.toFixed(2)}ms`,
      TTFB: `${ttfb.toFixed(2)}ms`,
      传输大小: `${(transfersSize / 1024).toFixed(2)}KB`
    });
  }
});

// 统计资源类型分布
const stats = resources.reduce((acc, r) => {
  acc[r.initiatorType] = (acc[r.initiatorType] || 0) + 1;
  return acc;
}, {});

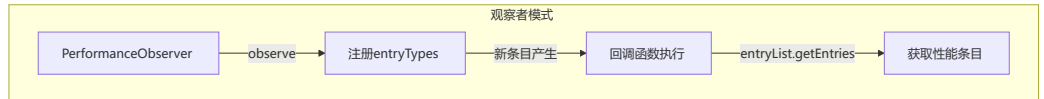
// 找出最大的资源
const sorted = [...resources].sort((a, b) => b.decodedBodySize - a.decodedBodySize);
console.log('最大资源:', sorted.slice(0, 5).map(r => ({
  url: r.name,
  size: `${(r.decodedBodySize / 1024).toFixed(2)}KB`,
  time: `${r.duration.toFixed(2)}ms`
})));

```

3 Performance Observer

💡 **要点：** 事件驱动替代轮询，支持 LCP/CLS/FID/Long Tasks 等现代 Web 指标，推荐使用 buffered: true 避免丢失早期条目。

Performance Observer 是订阅性能事件的最佳方式，替代 `performance.getEntries()` 的回调轮询模式。



完整示例:

```
// 创建观察者
const observer = new PerformanceObserver((list) => {
  const entries = list.getEntries();

  entries.forEach(entry => {
    if (entry.entryType === 'largest-contentful-paint') {
      console.log(`LCP: ${entry.startTime}ms`, entry.element);
    }

    if (entry.entryType === 'first-input') {
      console.log(`FID: ${entry.processingStart - entry.startTime}ms`, {
        类型: entry.name,
        延迟: entry.duration
      });
    }

    if (entry.entryType === 'layout-shift') {
      if (!entry.hadRecentInput) {
        console.log(`CLS: ${entry.value}`);
      }
    }

    if (entry.entryType === 'longtask') {
      console.warn(`长任务: ${entry.duration}ms`, entry.attribution);
    }
  });
});

// 注册需要观察的类型
observer.observe({ type: 'largest-contentful-paint', buffered: true });
observer.observe({ type: 'first-input', buffered: true });
```



```
observer.observe({ type: 'layout-shift', buffered: true });
observer.observe({ type: 'longtask', buffered: true });
observer.observe({ type: 'element', buffered: true });

// 断开观察
// observer.disconnect();
```

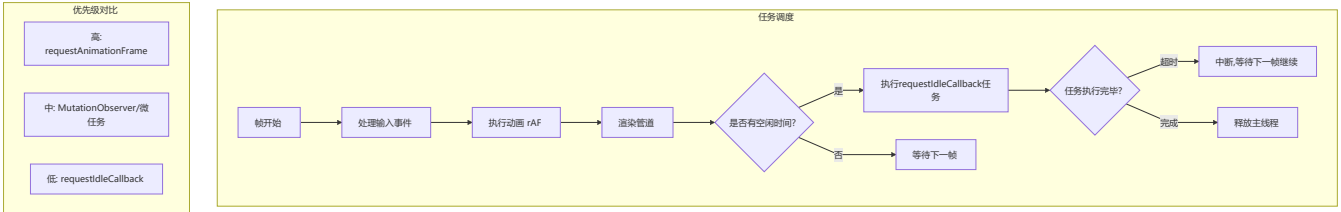
与回调方式的对比：

特性	PerformanceObserver	getEntries()轮询
实时性	事件驱动，即时	需要定时轮询
性能	无额外开销	轮询消耗CPU
新API支持	LCP/CLS/FID等	不支持新API
代码复杂度	声明式	命令式

4 requestIdleCallback

要点：在帧空闲期执行低优先级任务（埋点/日志/非关键计算），设置 timeout 兜底避免任务被饿死。

requestIdleCallback 允许在浏览器空闲时段执行低优先级任务，不影响关键渲染和交互。



使用示例：

```
// 注册空闲回调
requestIdleCallback(
  (deadline) => {
    // deadline.timeRemaining() 返回当前帧剩余时间(ms)
    while (deadline.timeRemaining() > 0 && tasks.length > 0) {
      const task = tasks.shift();
      task.execute();
    }

    // 如果还有任务未完成,继续注册
    if (tasks.length > 0) {
      requestIdleCallback(processTasks);
    }
  },
  { timeout: 2000 } // 超时：2秒后强制执行
);

// 实际应用：数据埋点/日志上报
function sendAnalytics(data) {
```

```
if ('requestIdleCallback' in window) {
  requestIdleCallback(() => {
    navigator.sendBeacon('/analytics', data);
  }, { timeout: 5000 });
} else {
  setTimeout(() => navigator.sendBeacon('/analytics', data), 0);
}
}

// 应用：延迟加载非关键计算
requestIdleCallback(() => {
  // 计算耗时统计报表
  const report = generateHeavyReport();
  updateUI(report);
});
```

与 requestAnimationFrame 的区别：

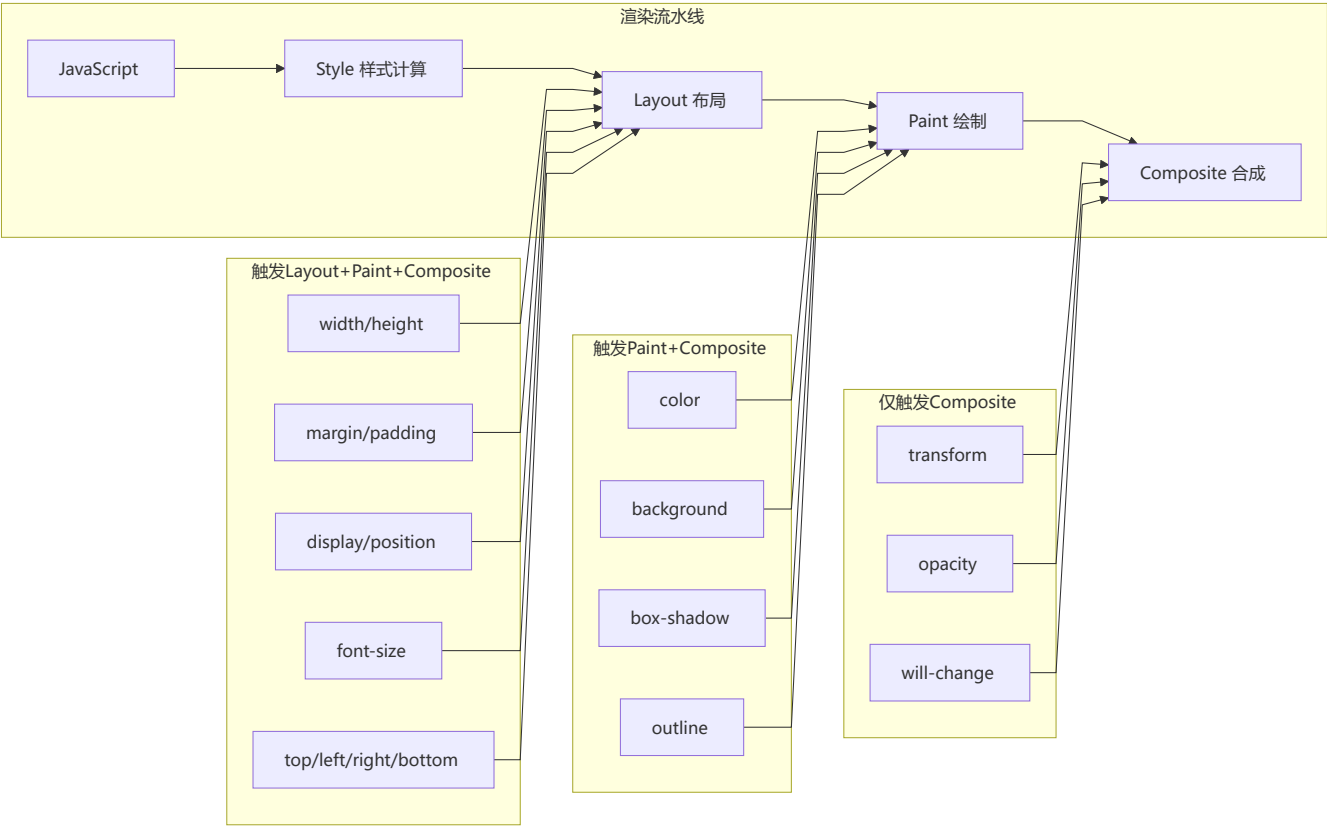
特性	requestIdleCallback	requestAnimationFrame
执行时机	帧空闲期	下一帧渲染前
优先级	最低	高
适用场景	非关键任务(埋点/日志)	动画/渲染更新
执行保证	不保证执行	每帧都会执行
参数	deadline.timeRemaining()	DOMHighResTimeStamp

⚡ 十三、浏览器渲染进阶

1 渲染流水线 (Rendering Pipeline)

💡 要点：渲染五阶段：JS → Style → Layout → Paint → Composite。transform/opacity 仅触发 Composite 性能最好，width/height 触发完整流水线。

现代浏览器的渲染过程是一个多阶段流水线，理解各阶段触发的 CSS 属性有助于针对性优化。



各阶段说明：

阶段	说明	触发示例
JavaScript	脚本执行，可能触发样式变更	<code>el.style.width = '200px'</code>
Style	计算元素最终样式	CSS 选择器匹配
Layout	计算元素几何位置和尺寸	<code>width/height/margin/padding/top/left</code>
Paint	绘制像素到图层	<code>color/background/border/box-shadow</code>
Composite	合成图层到屏幕	<code>transform/opacity/will-change</code>

💡 **Tip:** CSS 动画优先使用 `transform` 和 `opacity`，它们仅触发 Composite 阶段性能最优。避免频繁修改 `width/height/margin/padding` 等触发完整 Layout 的属性。使用 `will-change` 提前告知浏览器但切勿过度使用。

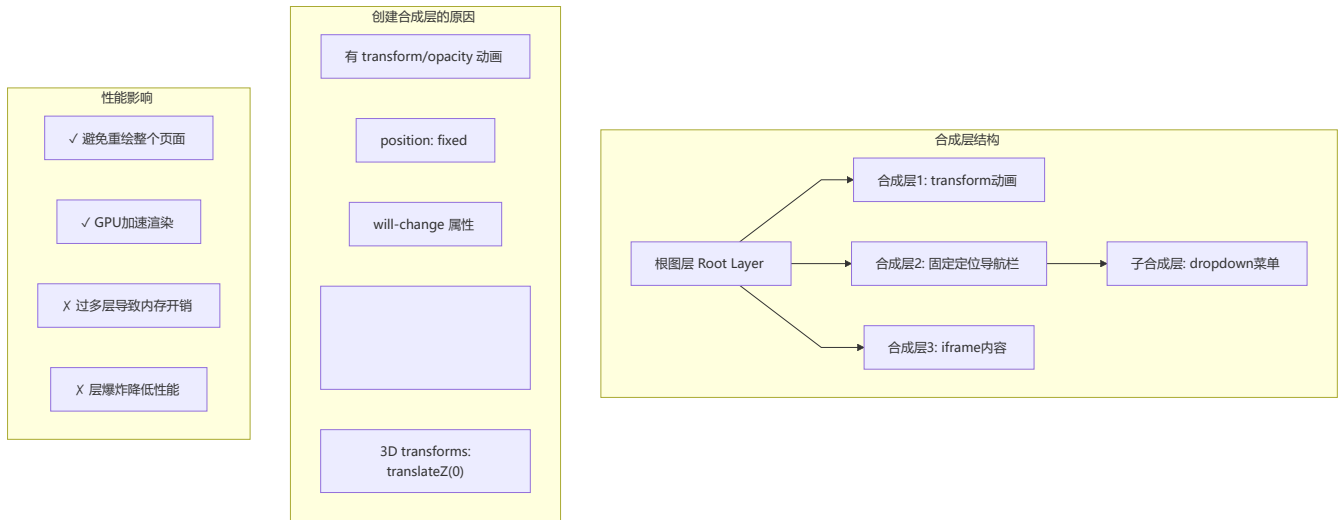
优化策略：

- 使用 `transform` 和 `opacity` 做动画（仅合成）
- 使用 `will-change` 提前告知浏览器（但勿过度使用）
- 批量修改 DOM，避免频繁触发 Layout
- 使用 `requestAnimationFrame` 批量更新

2 合成层 (Composite Layers)

💡 **要点：** 页面拆分为多个图层独立渲染，transform/opacity 动画在合成层上执行避免重绘。注意层爆炸问题，合理使用 will-change。

合成层是浏览器渲染优化的核心机制，将页面拆分为多个图层独立渲染并最终合成。



创建合成层的条件：

```
/* 1. 3D 或透视变换 */
.element { transform: translateZ(0); }

/* 2. 使用 will-change */
.element { will-change: transform, opacity; }

/* 3. 固定定位 */
.element { position: fixed; }

/* 4. opacity 动画 */
.element { transition: opacity 0.3s; }

/* 5. 滤镜效果 */
.element { filter: blur(5px); }

/* 6. 混合模式 */
.element { mix-blend-mode: multiply; }
```

层爆炸与层压缩：

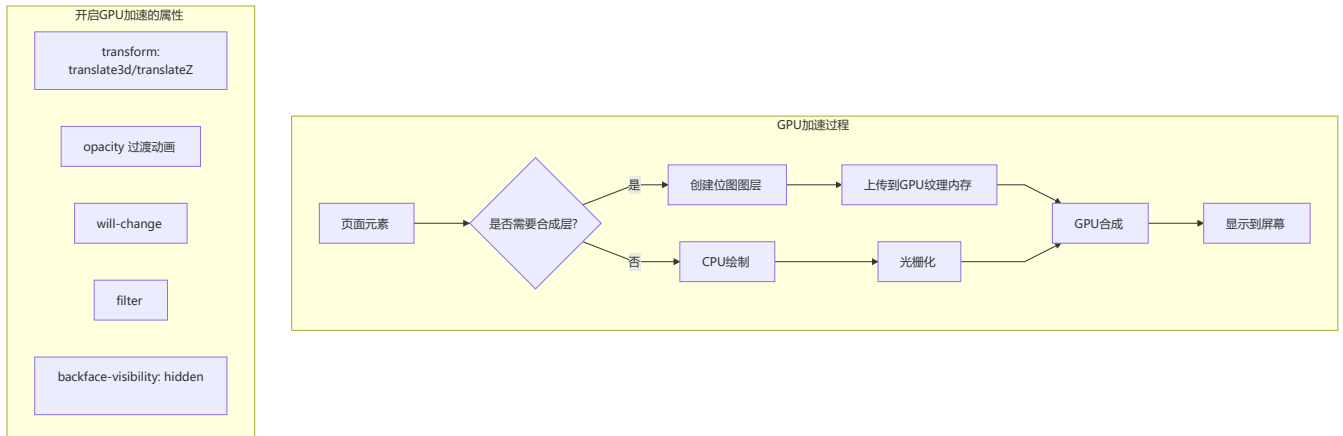
浏览器会尽量合并不必要的合成层以节省内存。过度使用 will-change 或 translateZ(0) 可能导致层爆炸。最佳实践：

- 只在需要动画时使用 will-change
- 动画结束后及时移除 will-change
- 使用 Chrome DevTools 的 Layers 面板监控合成层数量
- 避免在大量元素上同时创建合成层

3 硬件加速 (GPU Compositing)

💡 **要点：** GPU 合成提升动画帧率、减少 CPU 负载，但每个合成层占用 GPU 内存，过度使用（如全页面 `translateZ(0)`）会适得其反。

硬件加速指浏览器利用 GPU 进行合成操作，将某些渲染任务从 CPU 卸载到 GPU。



合理使用硬件加速：

```
/* 推荐：有动画的元素 */
.animated-card {
  will-change: transform;
  transition: transform 0.3s;
}

/* 避免：所有元素都开启 */
/* X 不要这样做 - 导致层爆炸 */
div { transform: translateZ(0); }

/* 更优方式：动画结束移除 */
.card {
  /* 初始不开启 */
}
.card.animating {
  will-change: transform;
}
.card.finished {
  will-change: auto; /* 恢复默认 */
}
```

优点：

- 合成速度更快，提升动画帧率
- 减少 CPU 负载，省电
- 避免重绘，提升滚动性能

注意事项：

- 每个合成层占用 GPU 内存（移动端尤其敏感）

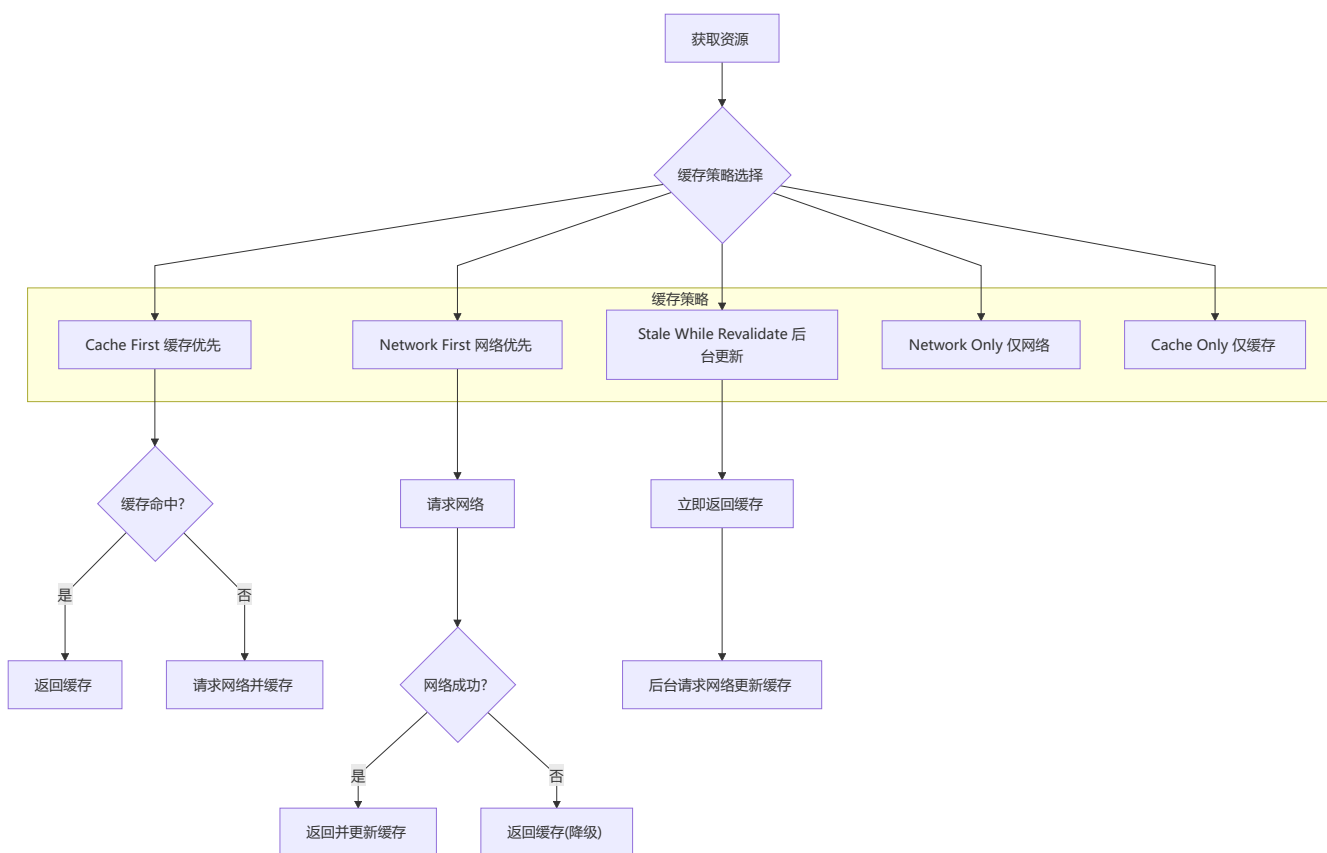
- 纹理上传 (CPU→GPU) 有额外开销
- 过多的合成层反而降低性能

十四、Web存储进阶

1 Cache API

💡 **要点：** Service Worker 的核心缓存机制，支持五种缓存策略：Cache First、Network First、Stale While Revalidate、Network Only、Cache Only。

Cache API 是 Service Worker 的核心组成部分，提供更灵活的请求/响应缓存机制。



Service Worker 结合使用：

```
// 安装阶段：预缓存关键资源
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open('v1').then(cache => {
      return cache.addAll([
        '/',
        '/index.html',
        '/styles/main.css',
        '/scripts/app.js',
        '/images/logo.png'
      ]);
    })
  );
});
```

```

});

// 拦截请求：缓存策略
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cachedResponse => {
      // 缓存优先 + 后台更新 (Stale-while-revalidate)
      const fetchPromise = fetch(event.request).then(networkResponse => {
        caches.open('v1').then(cache => {
          cache.put(event.request, networkResponse.clone());
        });
        return networkResponse;
      }).catch(() => cachedResponse);

      return cachedResponse || fetchPromise;
    })
  );
});

// 更新缓存
self.addEventListener('activate', event => {
  const cachewhitelist = ['v2'];
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.map(name => {
          if (!cachewhitelist.includes(name)) {
            return caches.delete(name);
          }
        })
      );
    })
  );
});

```

Cache API 与 HTTP Cache 的区别：

特性	Cache API	HTTP Cache
控制粒度	完全由 JS 控制	由 HTTP 头控制
缓存内容	Request/Response 对象	网络资源
更新机制	代码手动更新	Cache-Control 策略
生命周期	随 Service Worker	随浏览器策略
离线支持	是实现离线应用的基础	有限支持

2 Storage Foundation API

💡 **要点：** 查询存储用量和总配额，可请求持久化存储（用户清除数据时保留）。使用率超 80% 时应及时清理。

Storage Foundation API 提供存储容量查询和持久化请求，帮助管理浏览器的存储配额。

```
// 查询存储使用情况
async function checkStorage() {
  if ('storage' in navigator && 'estimate' in navigator.storage) {
    const estimate = await navigator.storage.estimate();

    const usedMB = (estimate.used / (1024 * 1024)).toFixed(2);
    const quotaMB = (estimate.quota / (1024 * 1024)).toFixed(2);
    const usagePercent = ((estimate.used / estimate.quota) * 100).toFixed(1);

    console.table({
      '已使用': `${usedMB} MB`,
      '总配额': `${quotaMB} MB`,
      '使用率': `${usagePercent}%`
    });

    // 接近配额时发出警告
    if (usagePercent > 80) {
      console.warn('存储空间即将用尽!');
    }
  }
}

// 请求持久化存储
async function requestPersistentStorage() {
  if ('storage' in navigator && 'persist' in navigator.storage) {
    const isPersisted = await navigator.storage.persist();

    if (isPersisted) {
      console.log('存储已持久化,用户清除数据时不会被删除');
    } else {
      console.log('存储未持久化,浏览器可能自动清除');
    }
  }

  // 检查当前持久化状态
  const persisted = await navigator.storage.persisted();
  console.log(`当前存储持久化状态: ${persisted ? '已持久化' : '未持久化'}`);
}
}
```

存储配额管理最佳实践：

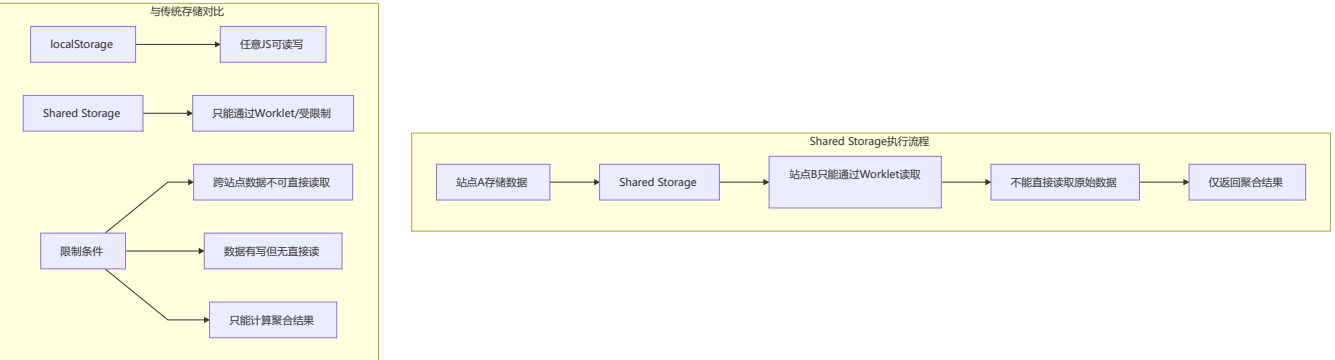
- 定期清理不再需要的数据
- 存储大文件时先检查剩余配额
- 使用 IndexedDB 的 `transaction.onabort` 处理配额超限错误

- 监听 `storageQuota` 相关事件（如 `quotaexceeded`）

3 Shared Storage API

💡 要点： Privacy Sandbox 的一部分，数据只能通过受限制的 Worklet 读取，不可直接读取原始数据。用于 A/B 测试分组和频率控制。

Shared Storage API 是 Privacy Sandbox 的一部分，允许跨站点共享数据但限制读取能力以保护隐私。



使用示例：

```
// 写入 Shared Storage
await window.sharedStorage.set('campaign-id', '12345', {
  ignoreIfPresent: true
});

// 通过 worklet 读取（受限）
class MyURLSelection {
  async run(urls, data) {
    const campaignId = await this.sharedStorage.get('campaign-id');
    const idx = campaignId ? parseInt(campaignId, 10) % urls.length : 0;
    return idx;
  }
}

register('select-url', MyURLSelection);
```

主要用例：

- **A/B 测试分组：** 跨站点保持一致的分组信息，而不暴露分组依据
- **频率控制：** 记录用户看到广告的次数，不暴露具体浏览历史
- **转化测量：** 统计广告点击后的转化，不追踪用户个体

与 localStorage 的区别：

特性	Shared Storage	localStorage
跨域访问	有限制(Worklet)	不允许
数据读取	仅通过受限制的 Worklet	任意 JS
隐私保护	内置隐私保护	无

特性	Shared Storage	localStorage
用途	广告/测量/实验	常规数据存储

4 OPFS (Origin Private File System)

💡 **要点：** 高效私有文件系统，支持同步访问（Web Worker 中）、零拷贝 ArrayBuffer 操作、硬盘直写，适合视频/游戏/WASM 大文件场景。

OPFS 是 File System Access API 的一部分，提供高效、私有的文件存储系统。

```
// 获取 OPFS 根目录
const root = await navigator.storage.getDirectory();

// 创建文件
async function createLargeFile() {
  const fileHandle = await root.getFileHandle('data.bin', { create: true });
  const accessHandle = await fileHandle.createSyncAccessHandle();

  // 写入 1MB 数据
  const buffer = new ArrayBuffer(1024 * 1024);
  const view = new Uint8Array(buffer);
  for (let i = 0; i < view.length; i++) {
    view[i] = i % 256;
  }

  const writtenBytes = accessHandle.write(buffer, { at: 0 });
  accessHandle.flush(); // 确保写入磁盘
  accessHandle.close();

  return writtenBytes;
}

// 读取文件
async function readLargeFile() {
  const fileHandle = await root.getFileHandle('data.bin');
  const file = await fileHandle.getFile();
  const sizeInMB = (file.size / (1024 * 1024)).toFixed(2);

  const accessHandle = await fileHandle.createSyncAccessHandle();
  const readBuffer = new ArrayBuffer(file.size);
  accessHandle.read(readBuffer, { at: 0 });
  accessHandle.close();

  return { buffer: readBuffer, size: file.size };
}

// 删除文件
async function cleanup() {
  await root.removeEntry('data.bin');
}
```

```
// 目录操作
async function directoryOps() {
  const dirHandle = await root.getDirectoryHandle('images', { create: true });
  const fileHandle = await dirHandle.getFileHandle('photo.jpg', { create: true });

  // 遍历目录
  for await (const [name, handle] of root.entries()) {
    console.log(name, handle.kind); // 'file' 或 'directory'
  }
}
```

性能优势：

- **同步访问**：在 Web Worker 中支持同步文件操作（createSyncAccessHandle）
- **零拷贝**：使用 ArrayBuffer 直接操作，避免内存复制
- **无需序列化**：无需将大对象序列化为文本
- **硬盘直写**：大文件直接在磁盘操作

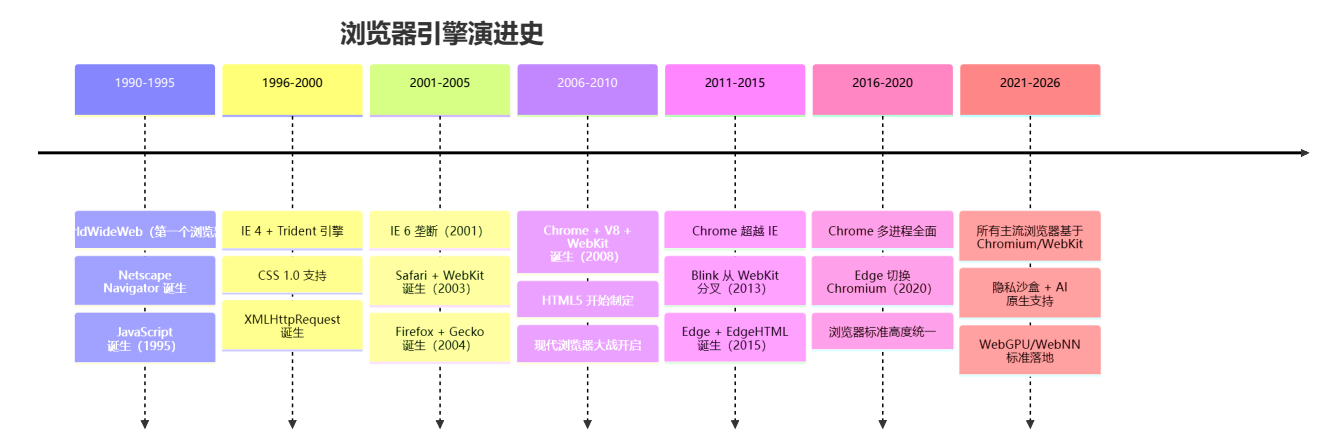
使用场景：

- 视频/音频编辑缓存
- WebAssembly 大型数据文件
- 游戏存档和资源包
- PDF/图像处理

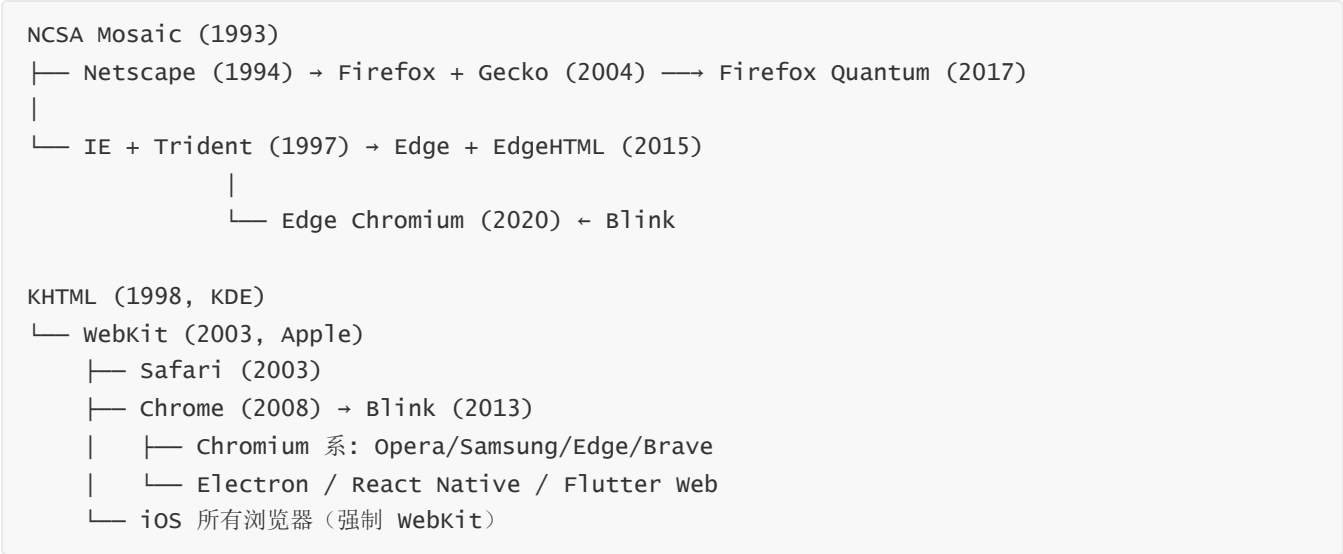
浏览器引擎演进史（1990—2026）

了解浏览器架构演变的来龙去脉，是理解现代浏览器原理的前提。

浏览器发展时间线



渲染引擎分支演化



现代浏览器架构对比

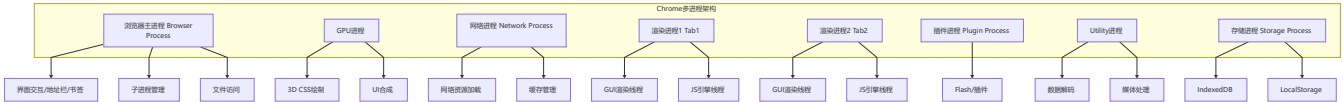
特性	Chrome (Chromium)	Firefox (Gecko)	Safari (WebKit)
多进程	✅ 全面 (Site Isolation)	✅ 全面	✅ 有限
GPU 合成	✅ Viz 合成器	✅ WebRender	✅
JS 引擎	V8	SpiderMonkey	JavaScriptCore
CSS 引擎	Blink	Stylo (Rust)	WebKit
沙箱	✅ 多重沙箱	✅ RLBox	✅
进程模型	1 Tab = 1 进程	1 Tab = 1 进程	Tab 分组共享
WASM	✅ 全面	✅ 全面	✅ 全面

🏠 十五、浏览器架构演进

1 Chrome 多进程架构

💡 **要点：**Chrome 多进程架构包含浏览器主进程、GPU 进程、网络进程、渲染进程（每个 Tab 独立）、插件进程等，提高稳定性和安全性。

Chrome 采用多进程架构，每个进程有独立的地址空间，互不影响。



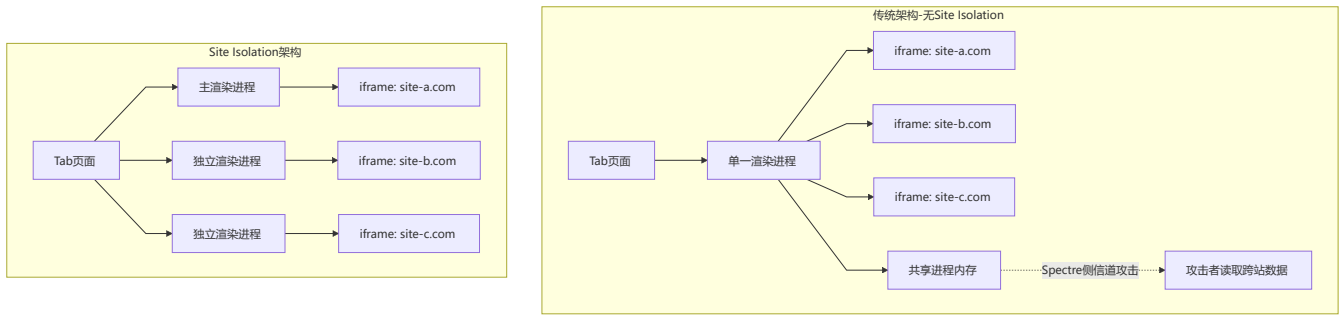
各进程职责：

进程	职责	数量
浏览器主进程	UI管理、进程调度、文件访问	1个
GPU进程	GPU加速、3D渲染合成	1个
网络进程	网络请求、缓存管理	1个
渲染进程	HTML/CSS/JS解析渲染（沙箱隔离）	每个Tab(含iframe)独立
插件进程	运行Flash等插件	每个插件独立
Utility进程	解码、媒体处理等辅助任务	按需创建
存储进程	IndexedDB/LocalStorage管理	1个

2 Site Isolation (站点隔离)

💡 **要点：** 跨站 iframe 运行在独立渲染进程中，进程级别内存隔离防御 Spectre 侧信道攻击，但增加约 10-20MB/iframe 的内存开销。

Site Isolation 是 Chrome 的安全架构，为每个跨站点 iframe 分配独立的渲染进程，防御 Spectre 类型攻击。



Site Isolation 的安全意义：

- 每个跨站 iframe 运行在独立的渲染进程中
- 进程级别的内存隔离，防御 Spectre/Meltdown 等侧信道攻击
- 攻击者无法通过共享内存读取其他站点的数据

内存开销：

- Site Isolation 增加了进程数量，每个跨站 iframe 占用约 10-20MB 额外内存
- Chrome 通过进程限制策略（如 per-process limit）控制总进程数
- 移动端 Chrome 可能禁用或限制 Site Isolation 以节省内存

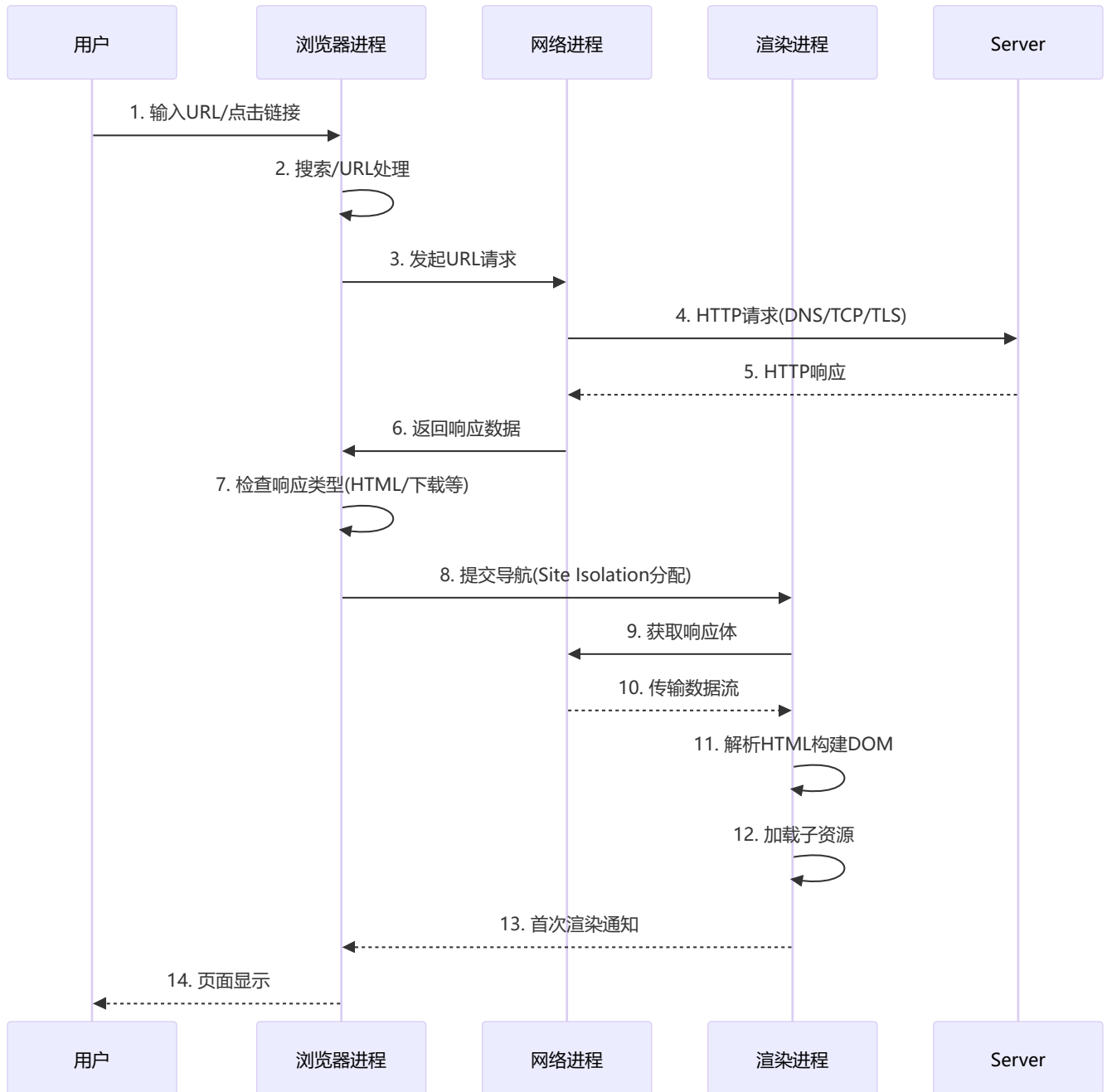
如何判断是否启用：

- 访问 `chrome://process-internals` 查看进程分配
- 任务管理器中查看站点所属进程（Chrome 自带任务管理器）

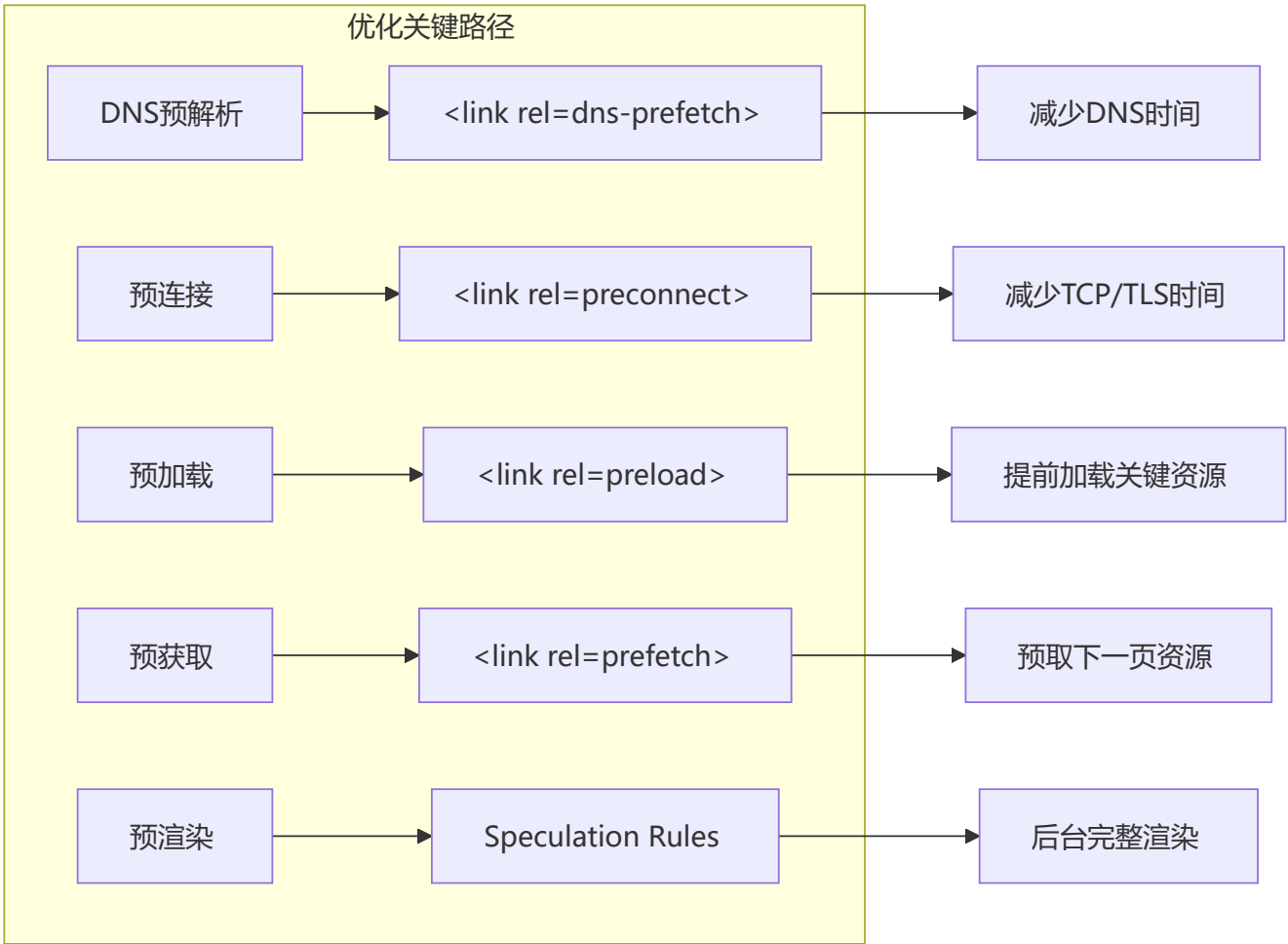
3 Navigation 架构

💡 **要点：** URL 输入到页面显示涉及浏览器进程、网络进程、渲染进程三者的协调，共 14 个步骤。DNS 预解析/预连接/预加载可优化关键路径。

从输入 URL 到页面显示，Chrome 的导航流程涉及多个进程的协调。



关键路径优化：



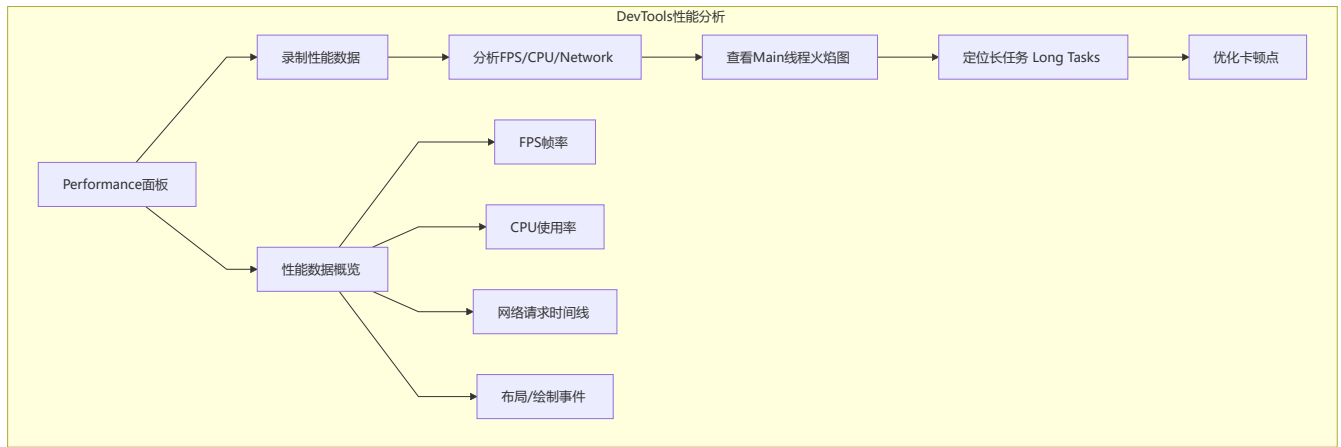
主要阶段耗时参考：

阶段	耗时(参考)	优化手段
DNS解析	20-120ms	DNS预解析、CDN
TCP连接	10-100ms	预连接
TLS握手	50-300ms	预连接、TLS1.3
TTFB	200-1000ms+	CDN、后端优化
DOM解析	100-500ms	精简HTML
子资源加载	500-3000ms+	懒加载、资源压缩

4 Chrome DevTools 使用技巧

💡 **要点：** 四大核心面板：Performance（分析长任务/布局阻塞）、Memory（查找内存泄漏/Detached DOM）、Network（请求耗时/瀑布图）、Application（SW/Cache/Storage 管理）。

Performance 面板：



关键使用技巧：

1. Performance 面板：

- 使用 `Ctrl+Shift+E` 开始/停止录制
- 查看 Main 火焰图识别长任务（红色标记）
- 关注 `Layout` 和 `Rendering` 阻塞事件
- 使用 Bottom-Up/Call Tree 分析耗时函数

2. Memory 面板：

- 使用 Heap Snapshot 查找内存泄漏
- 比对两次快照的 Retained Size 差异
- 使用 Allocation Timeline 记录分配过程
- 关注 Detached DOM 节点（DOM树外的元素）

```
// 常见内存泄漏检测
function findDetachedNodes() {
  // 在Heap Snapshot中搜索"Detached"
  // Detached HTMLDivElement 表示被删除但仍有引用的DOM节点
}
```

3. Network 面板：

- 使用 Waterfall 查看请求各阶段耗时
- 分析请求队列和阻塞时间
- 使用 Capture Screenshots 查看页面渲染过程
- 使用 `@media` 模拟不同网络条件（Slow 3G）

4. Application 面板：

- 查看 Service Worker 状态（离线/更新/激活）
- 管理 Cache Storage（清除/查看缓存内容）
- 检查 IndexedDB 存储结构
- 管理 Cookies、LocalStorage、SessionStorage
- 查看 Back/Forward Cache 测试结果

5. Layers 面板:

- 分析合成层数量和内存使用
- 检查哪些元素创建了独立层
- 定位层爆炸问题

🌟 十六、2025/2026 浏览器新特性

1 Speculation Rules API (预渲染 2.0)

💡 **要点:** 替代传统的 `<link rel="prefetch/preload">`, 通过 JSON 规则声明式配置预渲染/预获取策略, 支持后台完整渲染页面, 用户点击时瞬间展示。

Speculation Rules API 是新一代页面预加载方案, 比传统 Resource Hints 更强大:

```
<script type="speculationrules">
{
  "prerender": [
    {
      "source": "list",
      "urls": ["/next-page", "/about"],
      "eagerness": "moderate"
    }
  ],
  "prefetch": [
    {
      "source": "document",
      "where": {
        "and": [
          {"href_matches": "/*"},
          {"not": {"href_matches": "/admin/*"}}
        ]
      },
      "eagerness": "conservative"
    }
  ]
}
</script>
```

eagerness 级别:

- `eager`: 立即执行 (适合高确定性场景)
- `moderate`: 用户悬停链接时触发 (推荐)
- `conservative`: 用户滚动到视口内时触发

与传统方案对比:

特性	<code><link rel="prefetch"></code>	Speculation Rules
预渲染	❌ 仅下载资源	✅ 完整渲染页面

特性	<link rel="prefetch">	Speculation Rules
条件过滤	❌ 不支持	✅ href_matches 规则
触发策略	❌ 固定	✅ eagerness 分级
跨域支持	有限	更完善

2025 年状态：Chrome 120+ 稳定支持，Firefox 实验性支持，Safari 考虑中。Next.js 15+ 和 Astro 5 已内置集成。

2 View Transitions API 2.0

💡 **要点：**从 SPA 扩展到 MPA（多页应用），支持跨页面导航动画、共享元素过渡、自定义动画时间线，2025 年成为跨页面动效标准方案。

View Transitions API 在 2025 年迎来重大更新：

```
// SPA 内过渡（基础用法）
document.startViewTransition(() => {
  updateDOM();
});

// MPA 跨页过渡（2025 新增）
// 在 <head> 中声明
<meta name="view-transition" content="same-origin" />

// 共享元素过渡
.product-card {
  view-transition-name: product-card;
}

// 自定义动画
::view-transition-old(product-card) {
  animation: slide-out 0.3s ease-out;
}
::view-transition-new(product-card) {
  animation: slide-in 0.3s ease-out;
}
```

2025 年新特性：

- **MPA 支持：**跨页面导航自动应用过渡动画
- **多命名元素：**同时为多个元素设置 `view-transition-name`
- **动画时间线控制：**`animation-timeline: view()` 支持滚动驱动动画
- **Chrome DevTools 面板：**可视化调试过渡动画

3 第三方 Cookie 淘汰与 CHIPS

💡 要点: Chrome 125+ 默认禁用第三方 Cookie, 替代方案: CHIPS (Partitioned Cookie) 、 Privacy Sandbox API (Attribution Reporting、Topics) 、 First-Party Context。

2025 年 Chrome 正式淘汰第三方 Cookie, 前端需适配:

```
// CHIPS - 分区 Cookie (每个站点独立)
Set-Cookie: __Host-session=abc123; Path=/; Secure; SameSite=None; Partitioned

// Attribution Reporting API (广告归因)
// 替代第三方 Cookie 的转化追踪
const response = await fetch('/api/track', {
  headers: {
    'Attribution-Reporting-Register-Source': JSON.stringify({
      source_event_id: '123',
      destination: 'https://advertiser.com',
      expiry: 86400
    })
  }
});
```

替代方案对比:

方案	用途	隐私保护	状态
CHIPS	分区存储	✅ 每站点隔离	Chrome 稳定
Topics API	兴趣标签	✅ 本地计算	Chrome 稳定
Attribution Reporting	广告归因	✅ 聚合报告	Chrome 稳定
Protected Audience	重定向广告	✅ 本地拍卖	Chrome 稳定
First-Party Data	一方数据	✅ 用户授权	通用方案

4 2026 浏览器特性前瞻

特性	描述	浏览器支持
Declarative Shadow DOM	SSR 直接输出 Shadow DOM, 无需 JS 激活	Chrome/Edge 稳定
CSS Anchor Positioning	元素锚定到其他元素, 替代 JS 定位计算	Chrome 实验性
Popover API 增强	原生弹窗/提示框, 支持焦点管理和无障碍	Chrome/Firefox 稳定
WebSQL 替代: SQLite WASM	浏览器内运行 SQLite, 支持复杂查询	所有现代浏览器

特性	描述	浏览器支持
AI 推理 API: WebNN	原生神经网络推理, 支持 ONNX 模型	Chrome/Edge 实验性
Advanced Media Query	@media (prefers-reduced-motion: no-preference) 增强	全浏览器

WebNN API 示例 (2026 前沿) :

```
// 使用 webNN 运行 AI 模型 (替代 TensorFlow.js)
const context = await navigator.ml.createContext();
const graph = new MLGraphBuilder(context);

const input = graph.input('input', {type: 'float32', dimensions: [1, 224, 224, 3]});
const conv = graph.conv2d(input, weights, {padding: [1, 1, 1, 1]});
const output = graph.softmax(conv);

const model = await graph.build({output});
const result = await model.compute({input: imageData});
```

2026 浏览器性能趋势:

- Back/Forward Cache 成为标配, 页面秒级恢复
- Speculation Rules 替代传统预加载, LCP 优化 30%+
- INP 取代 FID 成为 Core Web Vitals 核心指标
- 隐私沙盒 全面替代第三方追踪
- AI 原生支持: WebNN、本地 LLM 推理成为新标准

 导航

 返回主指南	 下一章: 性能优化
---	---